GNU MCU Eclipse OpenOCD

ID: RK-KF-YF-91

Release Version: V2.0.0

Release Date: 2021-02-08

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2021. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces the debugging features of GNU MCU Eclipse OpenOCD.

Product Version

Chip Models	Kernel Version
RK3588/RK3568/RK3566/RK3399/RK3288/RK3368/ RK3326/RK3308/PX30/RV1108/RV1126/RV1109/ RV1106/RK2108/RK2206/RISCV	ALL

Target Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Hong Huibin	2020-04-21	Initial version
V1.1.0	Hong Huibin	2020-06-18	Major revisions to Chapter 2 (Operating System Environment); added Chapter 4; updated company name
V1.2.0	Hong Huibin	2020-08-07	Major revisions to OpenOCD command configuration and source code path configuration
V2.0.0	Hong Huibin	2021-02-08	Adjusted the document structure to optimize user onboarding and progressive learning

Table of Contents

GNU MCU Eclipse OpenOCD

- 1. Overview
- 2. Operating System Environment
  - 2.1 Windows
    - 2.1.1 Eclipse Software Package: openocd_eclipse.zip
    - 2.1.2 Run Eclipse; JRE Installation Required
    - 2.1.3 Enable Windows Telnet Feature
  - 2.2 Ubuntu 64-bit
    - 2.2.1 Eclipse package openocd_eclipse.tar.gz
    - 2.2.2 Software Installation
- 3. Quick Start
  - 3.1 Hardware Connection
    - 3.1.1 ARM 20-Pin Standard JTAG Connector
    - 3.1.2 TF to JTAG Adapter Board
    - 3.1.3 UART2 and JTAG Multiplexing
  - 3.2 Software Connection
    - 3.2.1 Run the software Eclipse_for_OpenOCD V1.0.exe
    - 3.2.2 Import Chip Configuration (Optional)
    - 3.2.3 Locate the target chip from the chip support list
    - 3.2.4 Connection Successful
    - 3.2.5 Connection Failure
      - 3.2.5.1 JTAG Adapter Not Recognized
      - 3.2.5.2 Chip Not Recognized
- 4. Basic Debugging Features
  - 4.1 Adjusting SWD/JTAG Speed
  - 4.2 Static Symbol Table Loading
  - 4.3 Dynamically Loading Symbol Tables
  - 4.4 Configure Source Code Path
  - 4.5 View Disassembly
  - 4.6 View Local Variables of Call Stack Functions
  - 4.7 View Global Variables
  - 4.8 Single-Step Debugging
  - 4.9 Setting Breakpoints
  - 4.10 View Memory Data
- 5. Advanced Debugging Features
  - 5.1 Register Groups
  - 5.2 Specify the Connected CPU
  - 5.3 Secure Debugging
  - 5.4 OpenOCD Command-Line Mode (Based on Command-Line Terminal)
  - 5.5 OpenOCD Command-Line Mode (Based on Eclipse)
  - 5.6 View Visualized Peripheral Registers
  - 5.7 View Visualized System Control Registers
  - 5.8 Export Data from Memory to File
  - 5.9 Import Data from File to Memory
  - 5.10 Compare File Data with Memory Data
- 6. Practical Application Scenarios
  - 6.1 Debugging CPU Hang Issues
  - 6.2 RT-Thread Debugging Instructions
  - 6.3 Single-Step Debugging of Linux
    - 6.3.1 Single-Core Kernel Phase
    - 6.3.2 Kernel Multi-core Phase

6.4 Bare-metal Debugging
6.4.1 Create Project
6.4.2 Load and Run Firmware
6.5 Using kgdb
6.5.1 Purpose of kgdb
6.5.2 Enabling kgdb
6.5.3 kgdb Configuration
6.6 Debugging in aarch64 32-bit Mode

1. Overview

Debugging Architecture: Eclipse CDT + GNU MCU Eclipse OpenOCD + Eclipse + GDB + OpenOCD + FTDI/J-Link + SoC

- Eclipse CDT (C/C++ Development Tooling): C/C++ development tools
- GNU MCU Eclipse OpenOCD: An open-source plugin primarily facilitating interaction between CDT, GDB, and OpenOCD
- Eclipse: A powerful tool capable of integrating various plugins; ARM DS-5 is also based on it
- GDB: GNU Debugger
- OpenOCD: An open-source debugging software compatible with various SWD/JTAG adapters, supporting architectures such as ARM and RISC-V
- FTDI: Utilizes the FT232H, a USB-to-SWD/JTAG chip that serves as an SWD/JTAG adapter, offering high speed and stability

2. Operating System Environment

2.1 Windows

2.1.1 Eclipse Software Package: openocd_eclipse.zip

Extract the archive and navigate to the root directory:

- Execute "Eclipse_for_OpenOCD V1.0.exe" to launch the Eclipse software.
- RK Directory:
- eclipse-workspace: Working directory; Eclipse defaults to this folder.
- example: Related examples.
- tools: Open-source tools such as GDB and JDK
- Practical Videos: Quick start guide
- OpenOCD: Files related to OpenOCD
- SVD (CMSIS System View Description format): Primarily used for viewing chip Registers
- doc: Usage help documentation

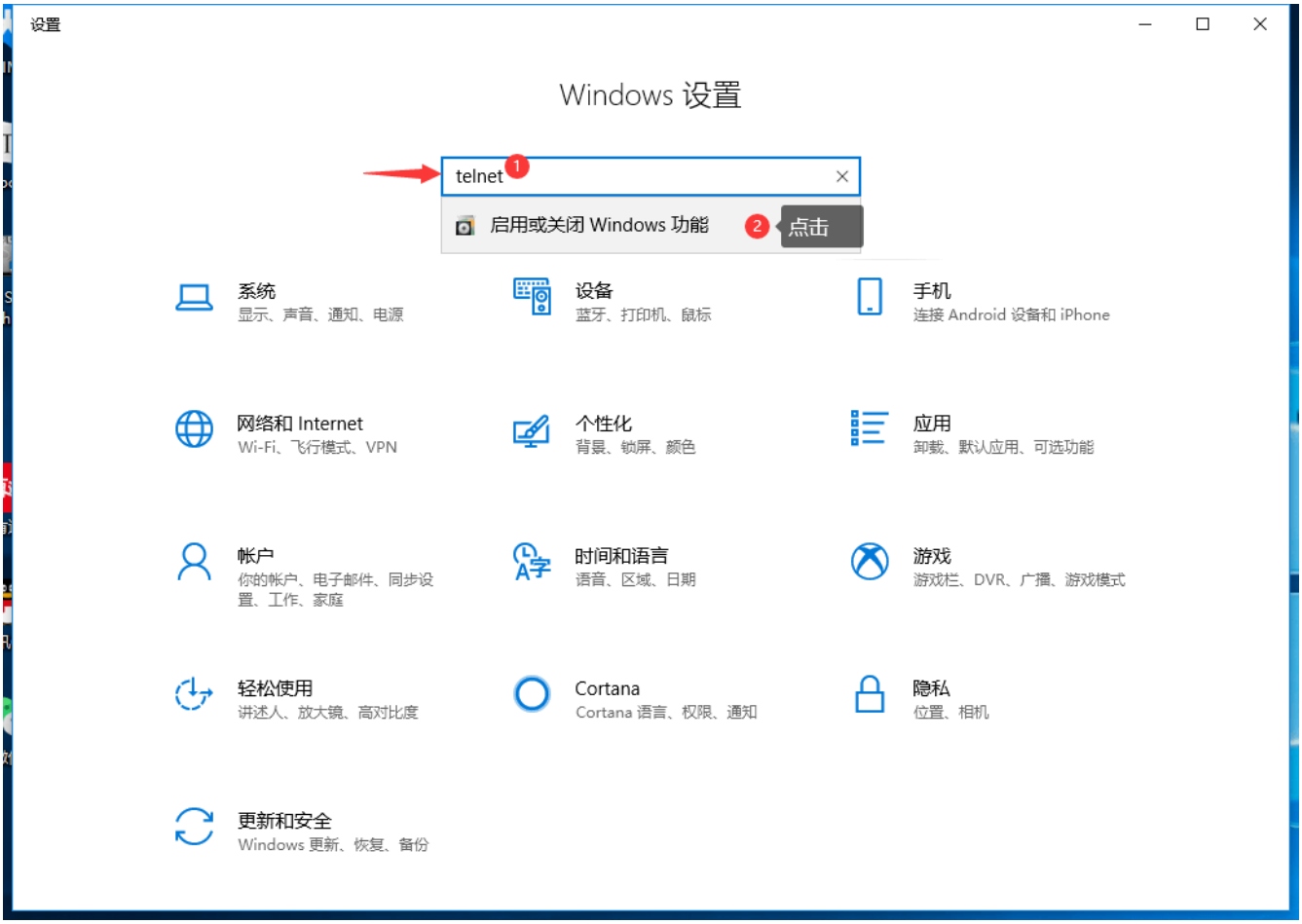
2.1.2 Run Eclipse; JRE Installation Required

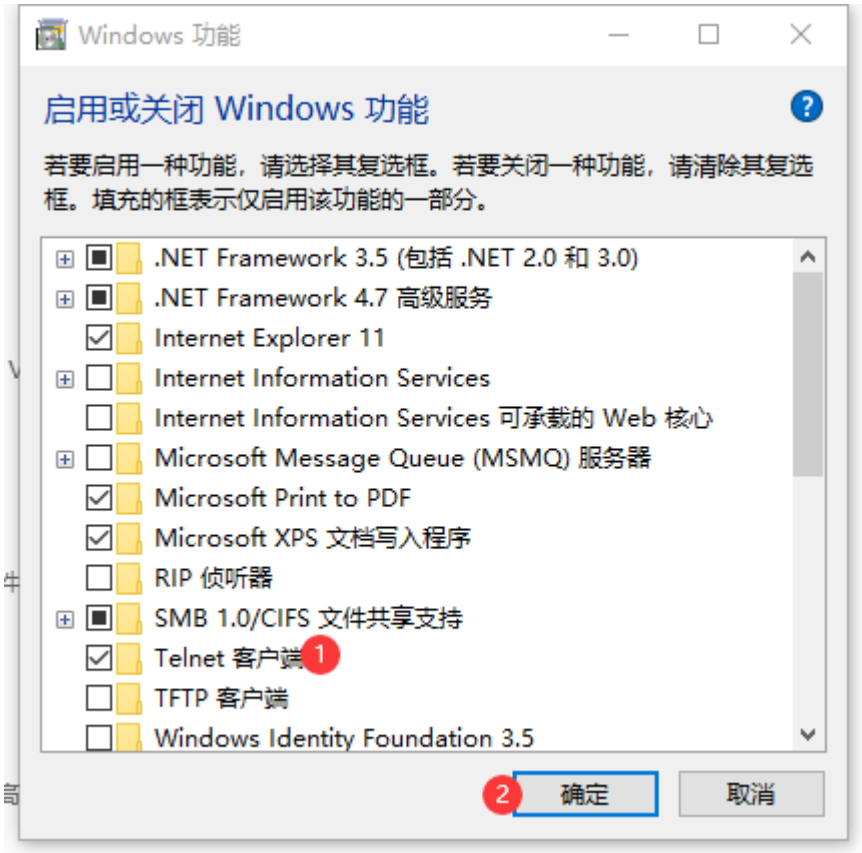
RK\tools\jdk_8.0.1310.11_64.exe

2.1.3 Enable Windows Telnet Feature

Telnet is used to access the OpenOCD command-line mode.

The method to enable Telnet may vary across different Windows versions; please search online for specific instructions. The following section primarily covers Windows 10:





2.2 Ubuntu 64-bit

2.2.1 Eclipse package openocd_eclipse.tar.gz

Extract openocd_eclipse.tar.gz:

```
tar -xzvf openocd_eclipse.tar.gz
```

Enter the home directory:

- eclipse: Execute this file to launch the Eclipse IDE.
- RK Directory:
- eclipse-workspace: Working directory; Eclipse defaults the workspace to this folder.
- example: Related examples.
- OpenOCD: Files related to OpenOCD.
- SVD (CMSIS System View Description format) is primarily used to view chip Registers
- tools: Open-source related tools, such as GDB
- doc: Usage documentation
- Practical videos: Quick start guide

2.2.2 Software Installation

- Running Eclipse requires JRE installation.

```
sudo add-apt-repository ppa:openjdk-r/ppa
sudo apt-get update
sudo apt-get install openjdk-8-jre  # This is not strictly required
```

- Running OpenOCD requires libusb.

```
sudo apt-get install libusb-1.0-0-dev
sudo apt-get install libftdi-dev
```

- Copy USB device information

```
sudo cp RK/OpenOCD/drivers/99-openocd.rules /etc/udev/rules.d/
sudo cp RK/OpenOCD/drivers/60-openocd.rules /etc/udev/rules.d/
```

- Install the open-source disassembly tool Capstone.

```
sudo apt-get install libcapstone-dev
```

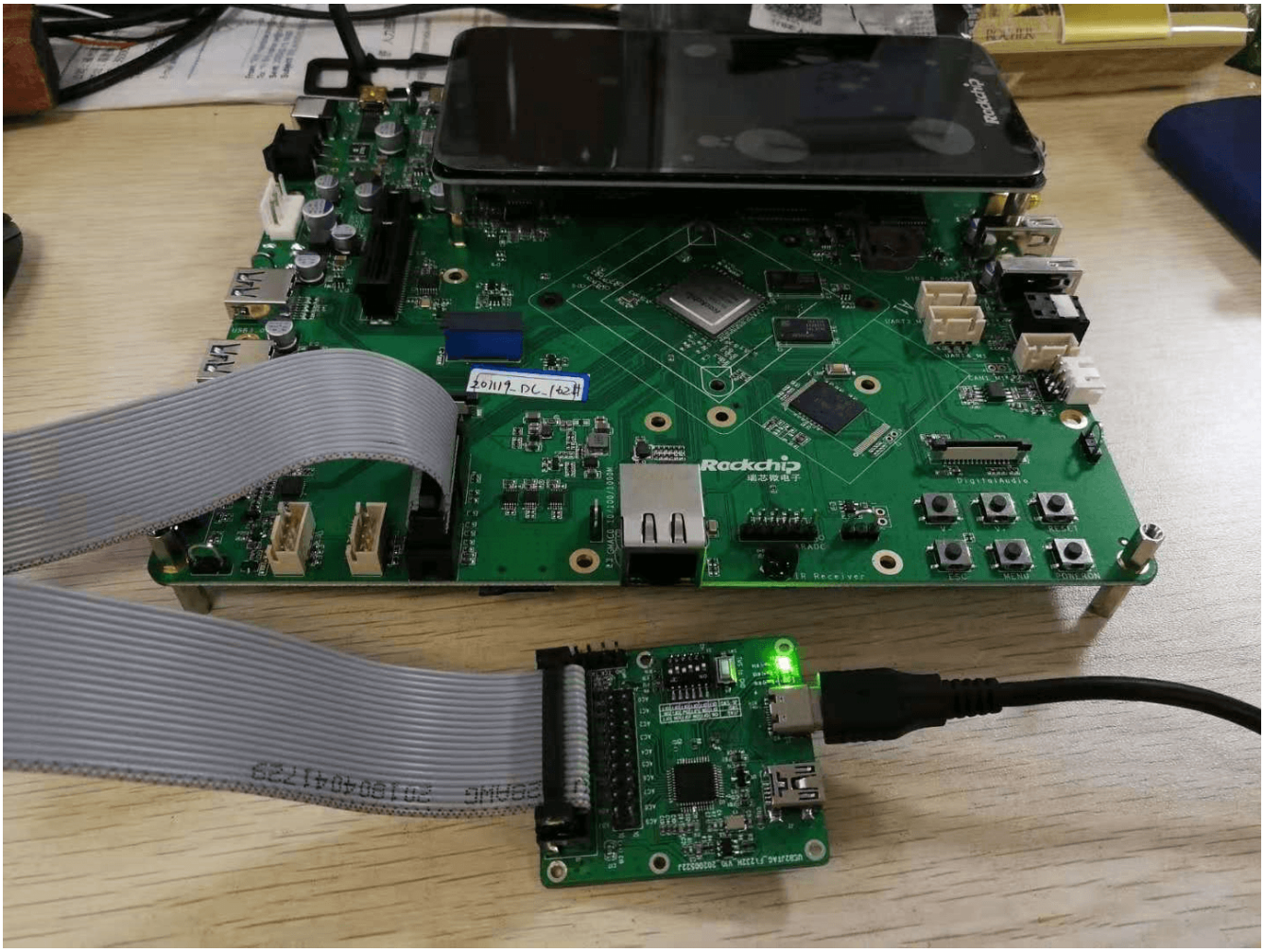
Note: Tested successfully on Ubuntu 16.04 and Ubuntu 18.04.

3. Quick Start

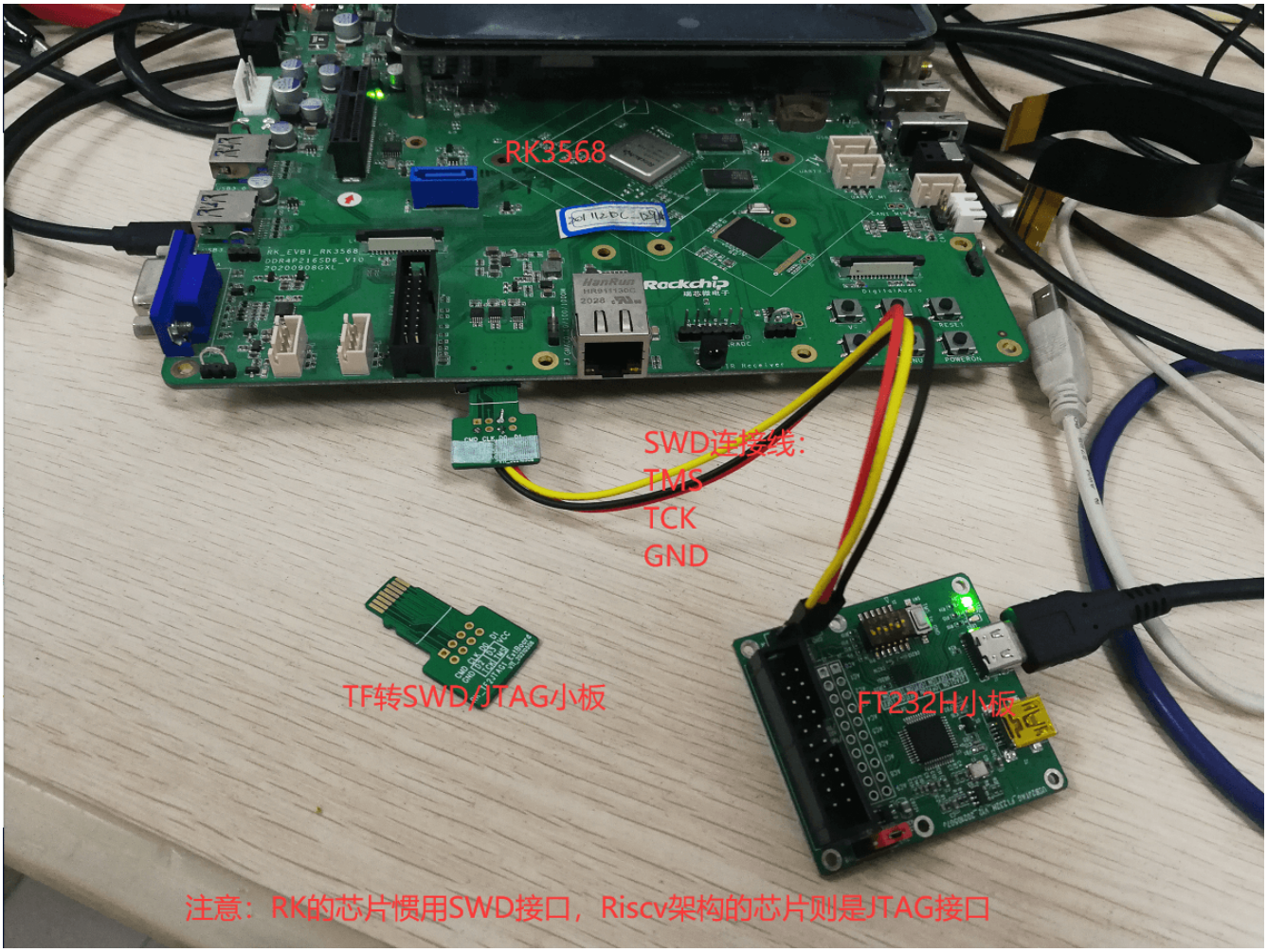
The UI is essentially the same on both Ubuntu and Windows. Since most users are accustomed to Windows, this guide focuses primarily on Windows.

3.1 Hardware Connection

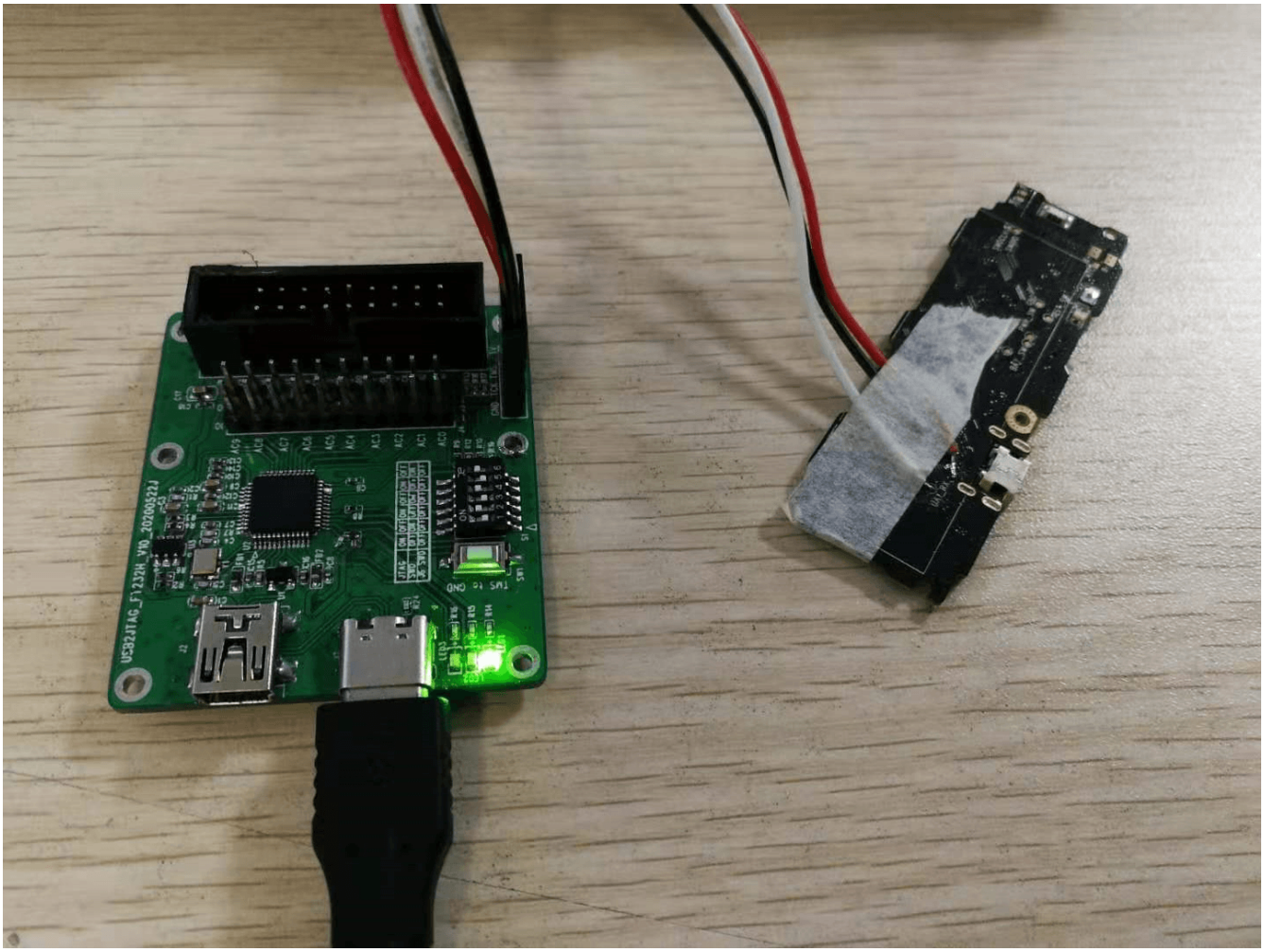
3.1.1 ARM 20-Pin Standard JTAG Connector



3.1.2 TF to JTAG Adapter Board

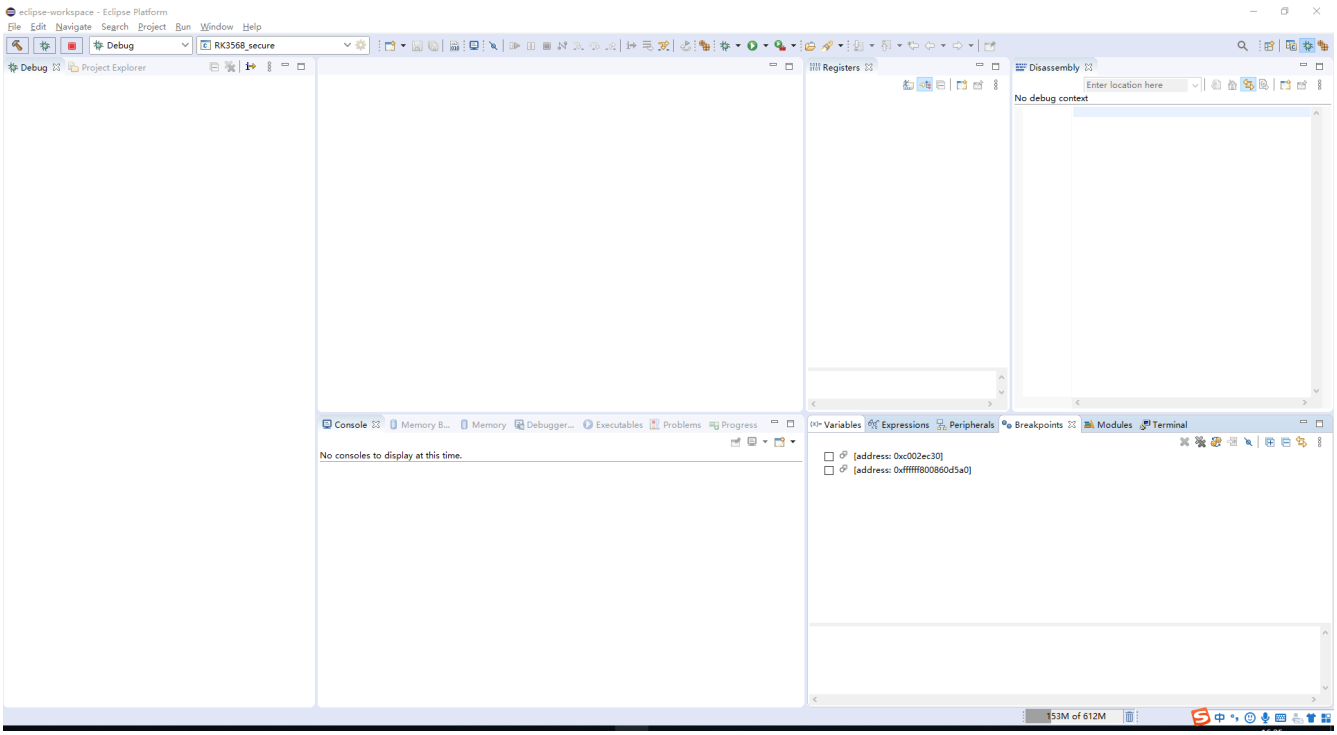


3.1.3 UART2 and JTAG Multiplexing



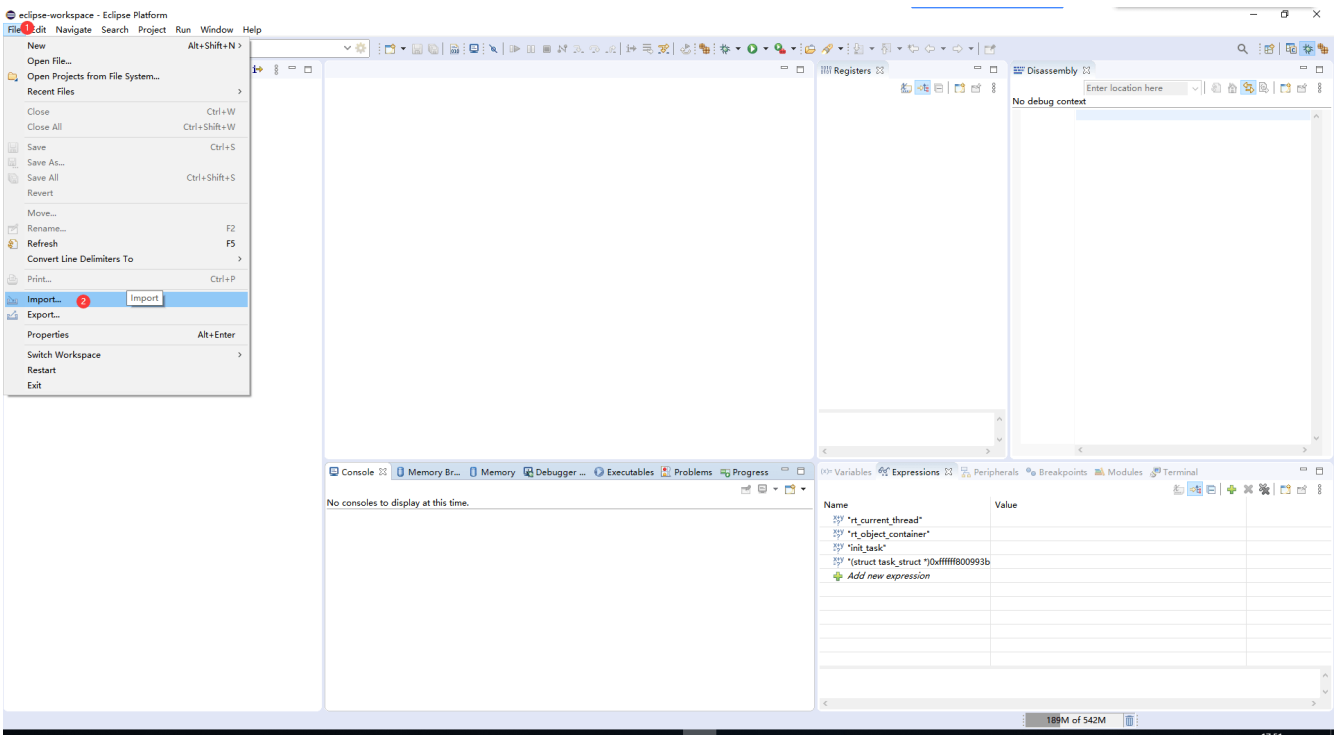
3.2 Software Connection

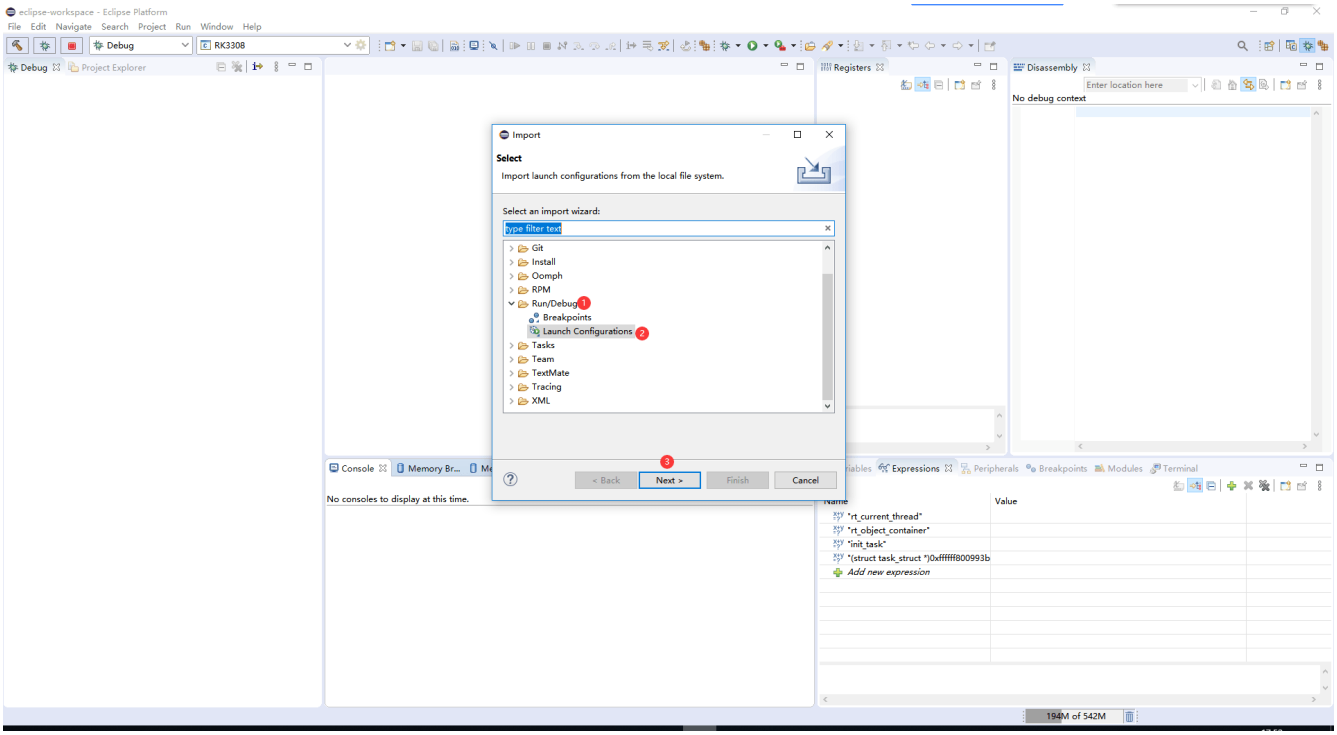
3.2.1 Run the software Eclipse_for_OpenOCD V1.0.exe

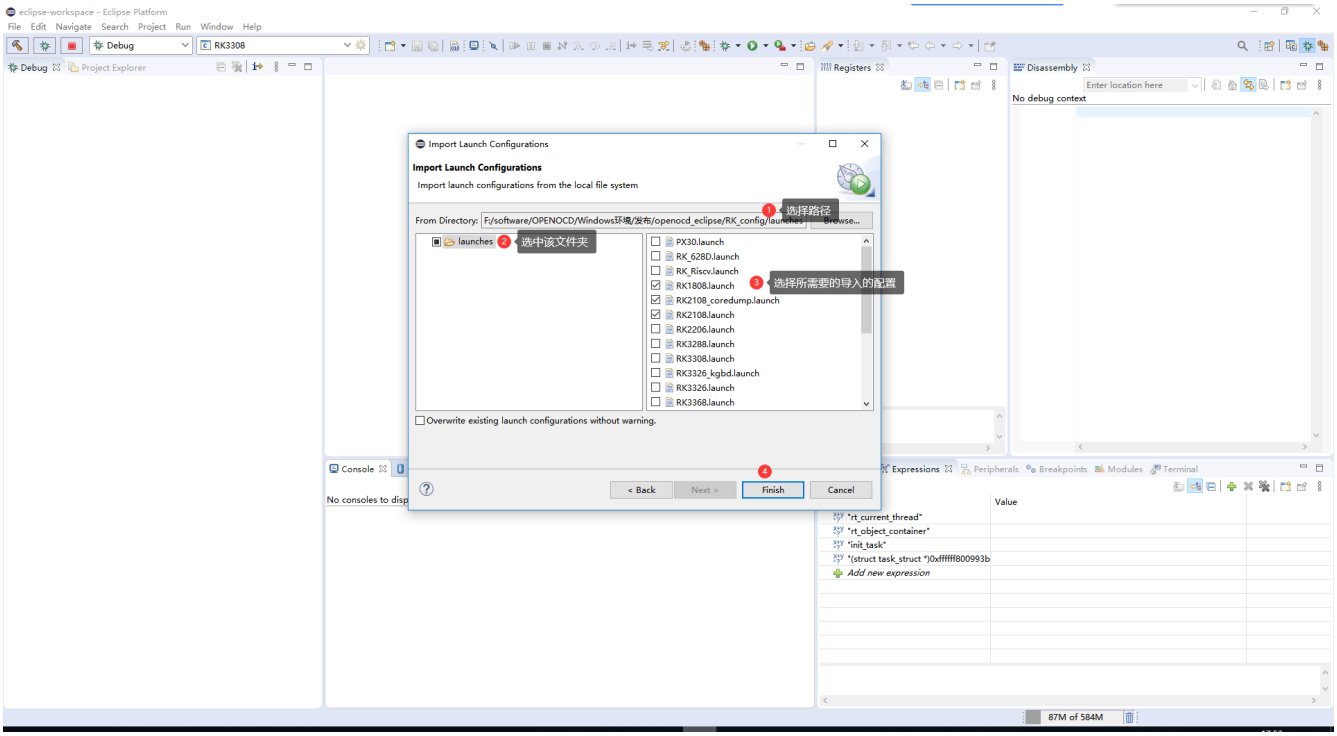


3.2.2 Import Chip Configuration (Optional)

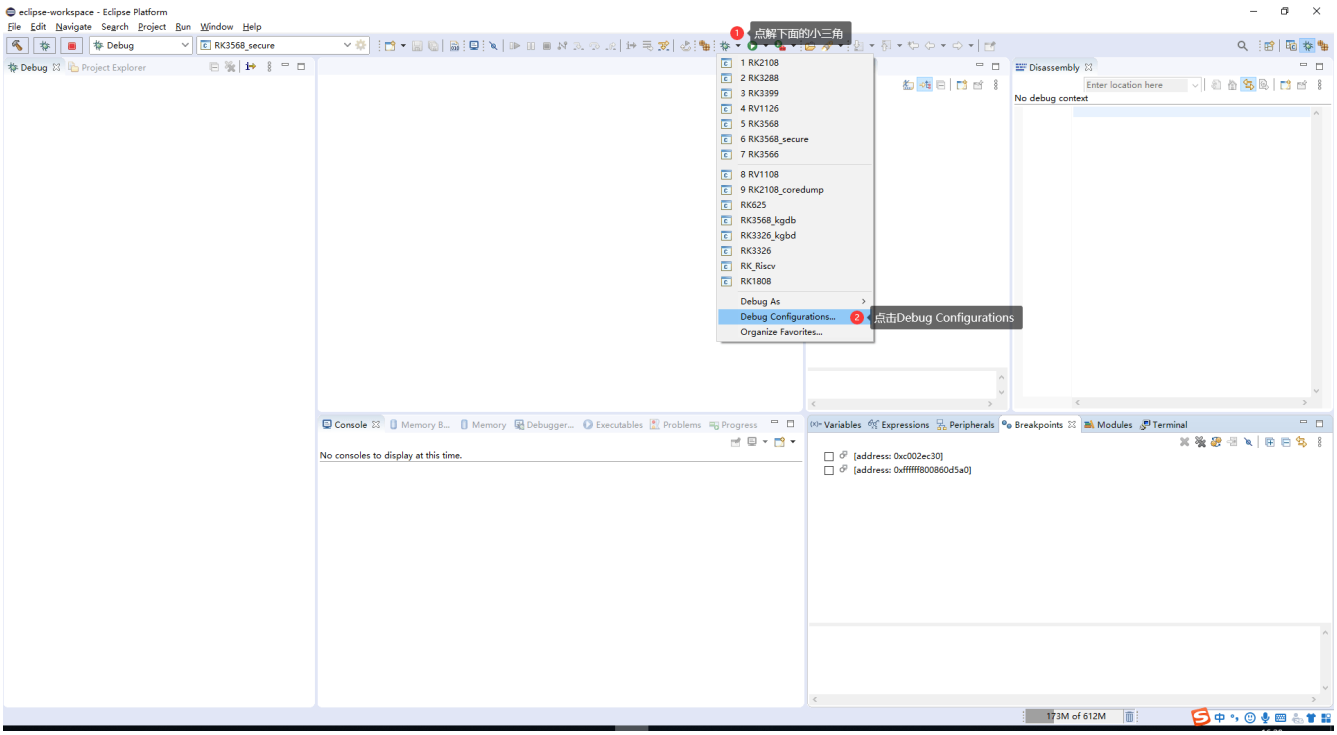
The software package normally includes the chip configuration. However, as chip configurations are continuously updated, modified, or added, it may be necessary to import the chip configuration.

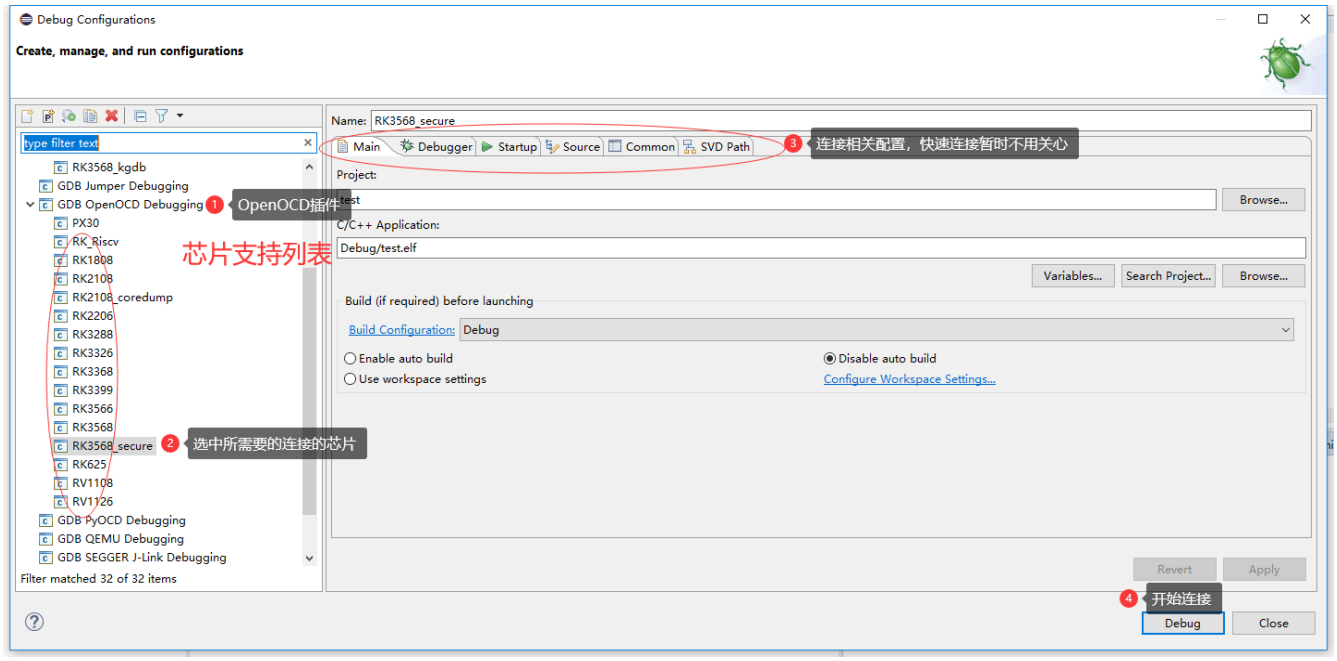






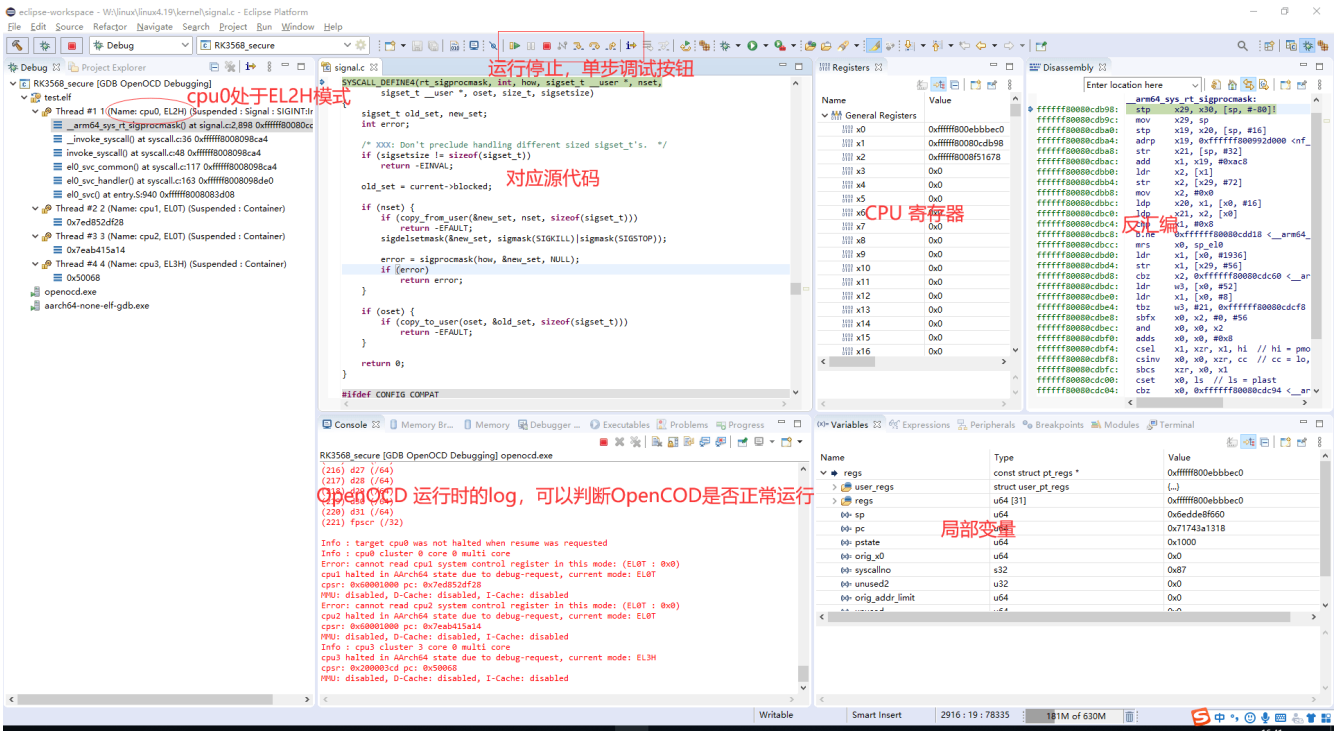
3.2.3 Locate the target chip from the chip support list





3.2.4 Connection Successful

The result after stopping the CPU is as follows:

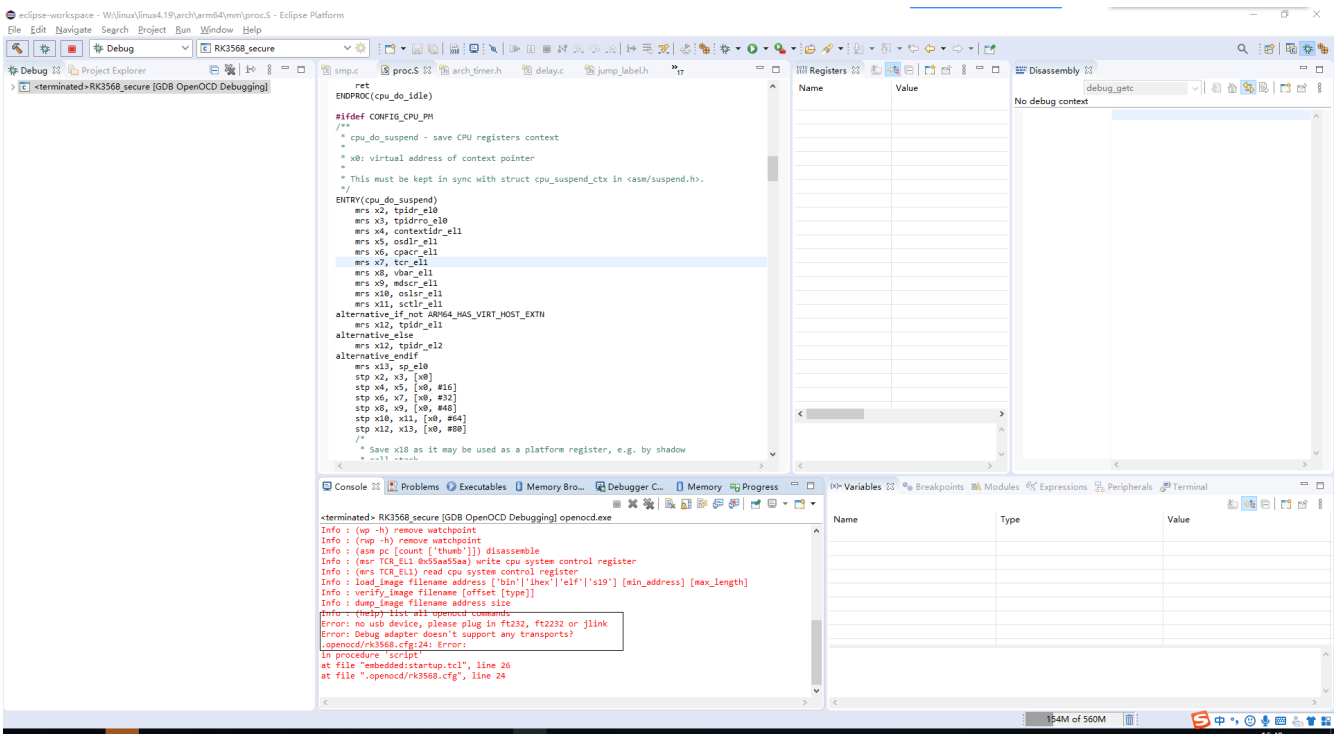


3.2.5 Connection Failure

3.2.5.1 JTAG Adapter Not Recognized

Verify the adapter driver or check whether the adapter functions correctly. Refer to the document "Rockchip_Developer_Guide_FT232H_USB2JTAG.pdf".

Note: OpenOCD can automatically recognize ft232h, ft2232h, and jlink adapters by default.



3.2.5.2 Chip Not Recognized

Verify that the chip pin contacts are normal, ensure the chip's JTAG IOMUX is enabled, or reduce the JTAG TCK frequency.

- The connection is OK, but the CPU is in an incorrect state

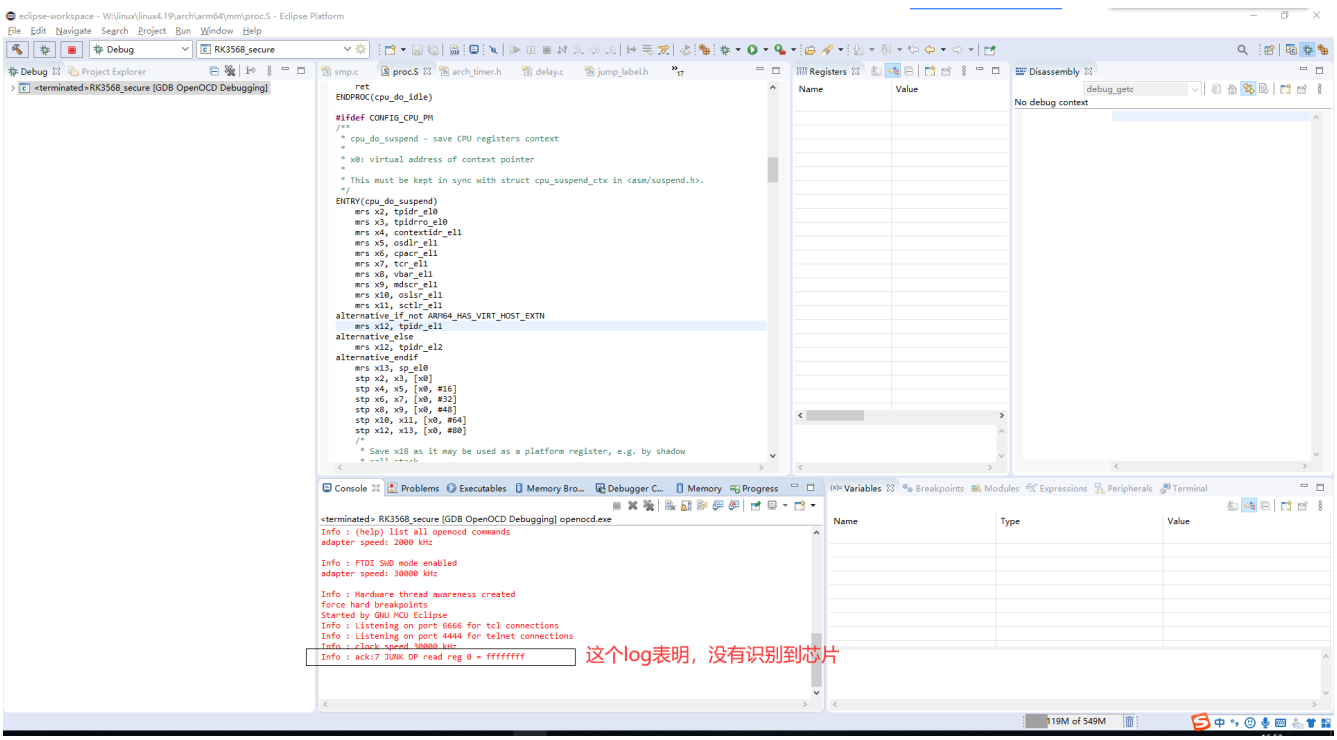
This indicates that the hardware connection is normal and JTAG has detected the device, but the chip state is abnormal; debugging cannot proceed in this case.

- please check hardware, make sure swd/jtag iomux is correct, and the pins connection is ok.
- please check hardware, the swdio(tms) pin is low level.

The above two cases indicate hardware connection issues, possibly due to incorrect IOMUX configuration or poor pin contact.

- Please set adapter speed to 1000 (1MHz) and try again

Upon encountering this log, reduce the JTAG frequency and attempt to connect again.

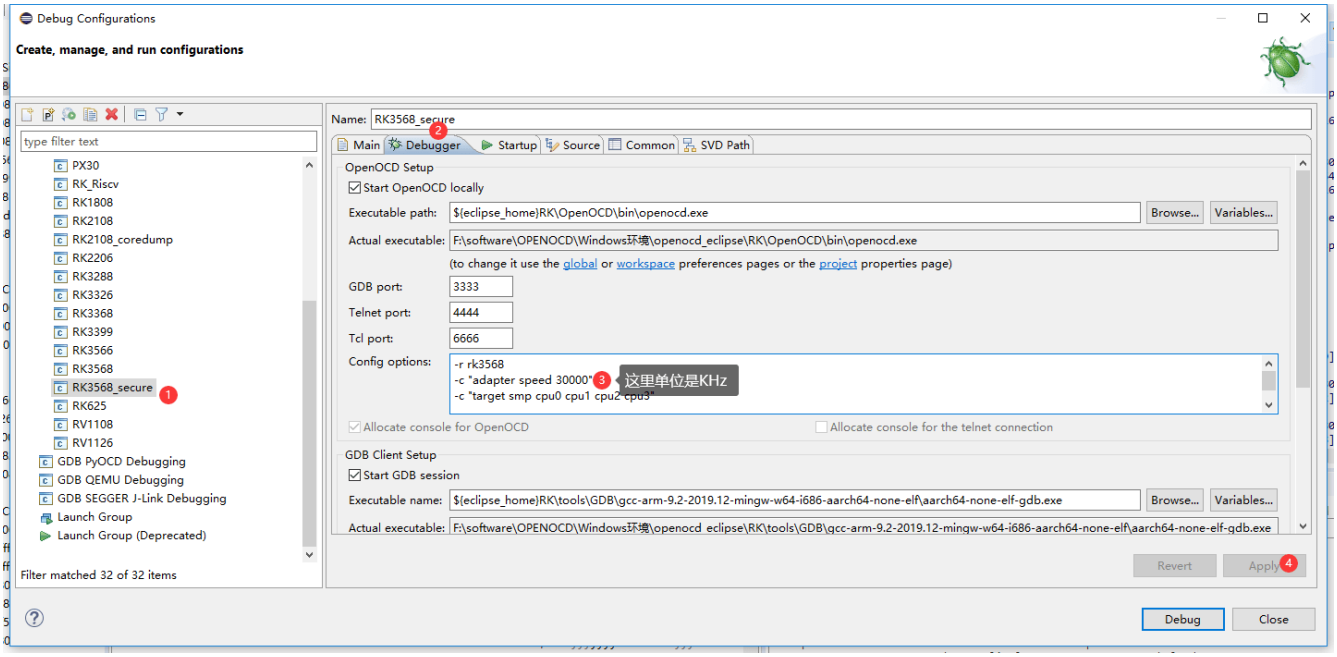


4. Basic Debugging Features

4.1 Adjusting SWD/JTAG Speed

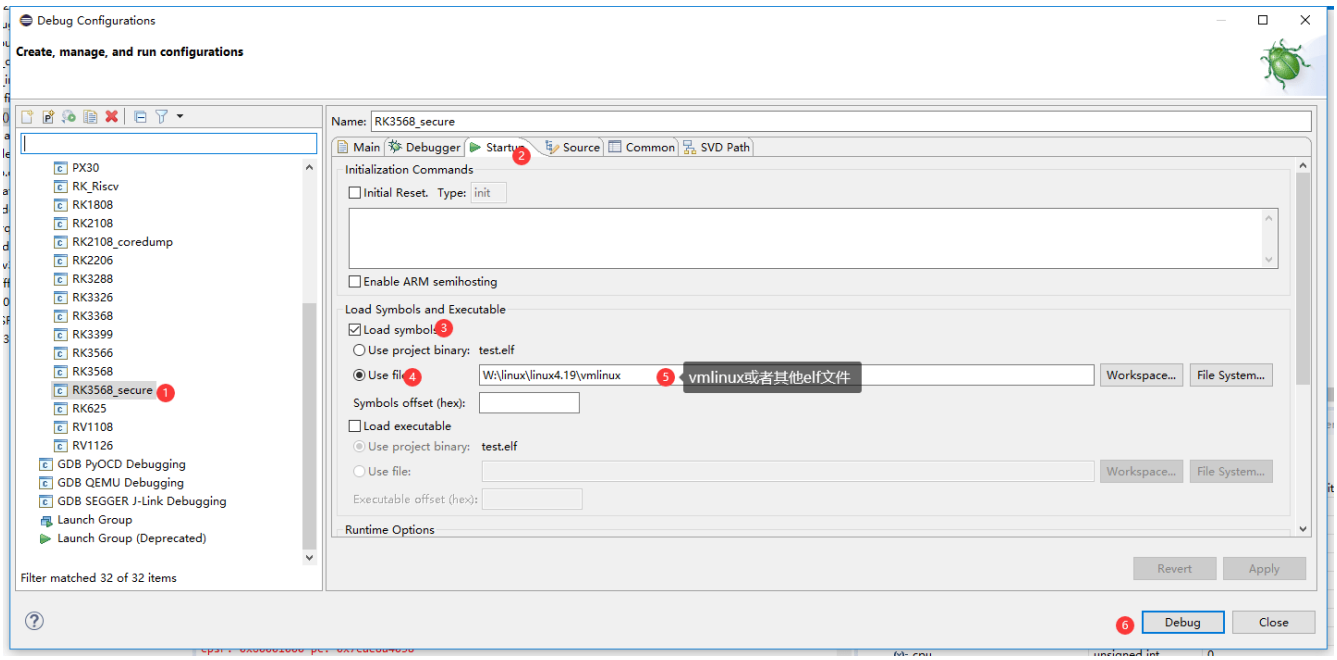
If hardware limitations cause SWD/JTAG communication failures, reduce the TCK speed, for example to 1000 kHz.

The maximum speed is 30000 kHz; 15000 kHz or 7500 kHz is recommended.



4.2 Static Symbol Table Loading

Static symbol table loading must be completed before linking, and only one symbol table can be loaded.



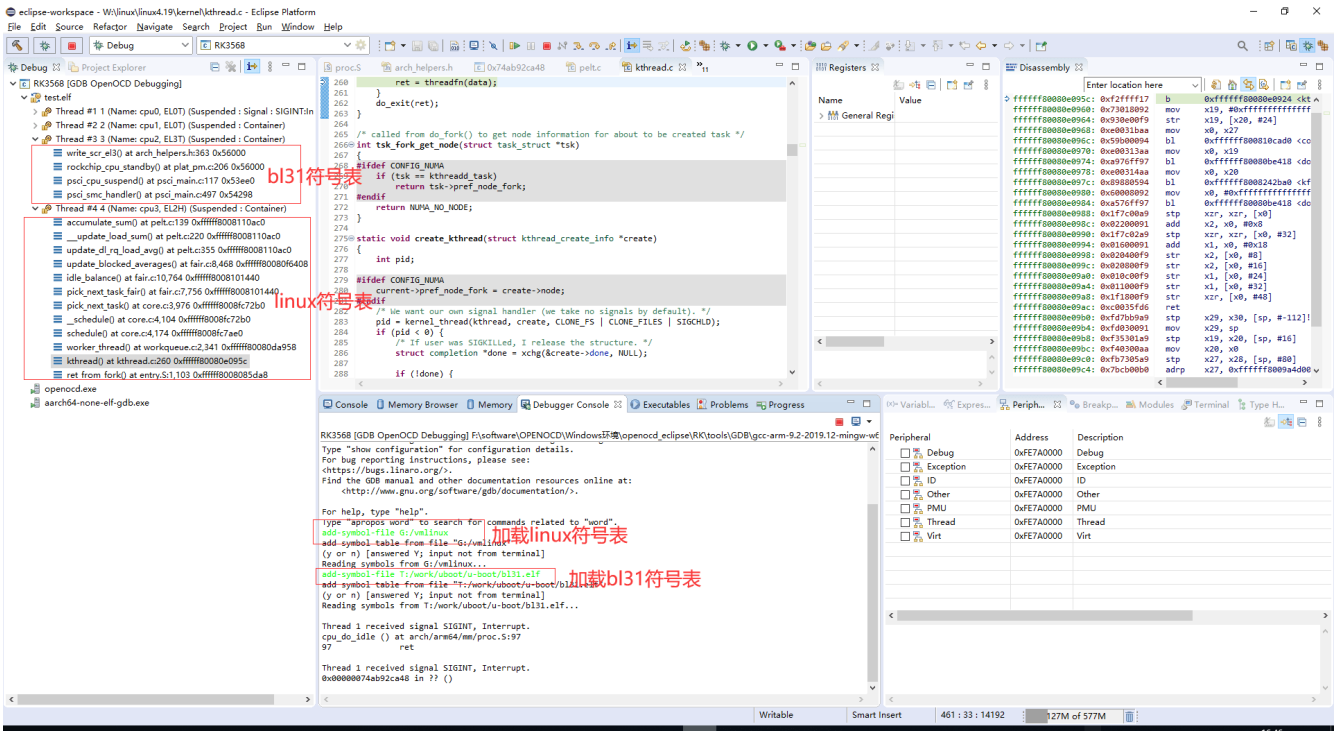
4.3 Dynamically Loading Symbol Tables

In the Debugger Console window, which is the interface for executing GDB commands, run the `add-symbol-file` command to load symbol tables. Multiple symbol tables can be loaded simultaneously.

```
Load the Linux symbol table: add-symbol-file G:/vmlinux
Load the bl31 symbol table: add-symbol-file T:/work/uboot/u-boot/bl31.elf
Load the TPL symbol table: add-symbol-file T:/work/uboot/u-boot/tpl/u-boot-tpl
Load the SPL symbol table: add-symbol-file T:/work/uboot/u-boot/spl/u-boot-spl
Load the U-Boot symbol table: add-symbol-file T:/work/uboot/u-boot/u-boot
```

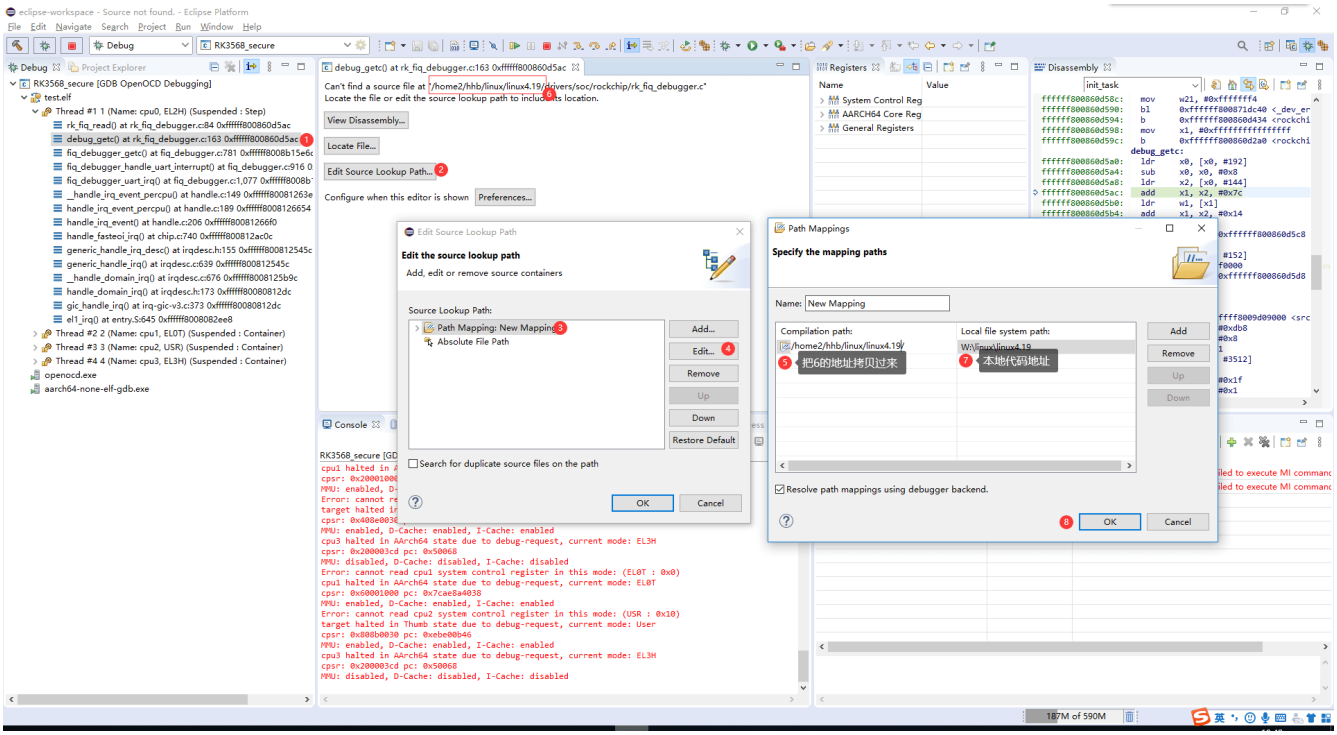
Note: In the Windows environment, replace "\" with "/".

Additionally, after executing the load command, single-step execution is required for the function call stack to be displayed.

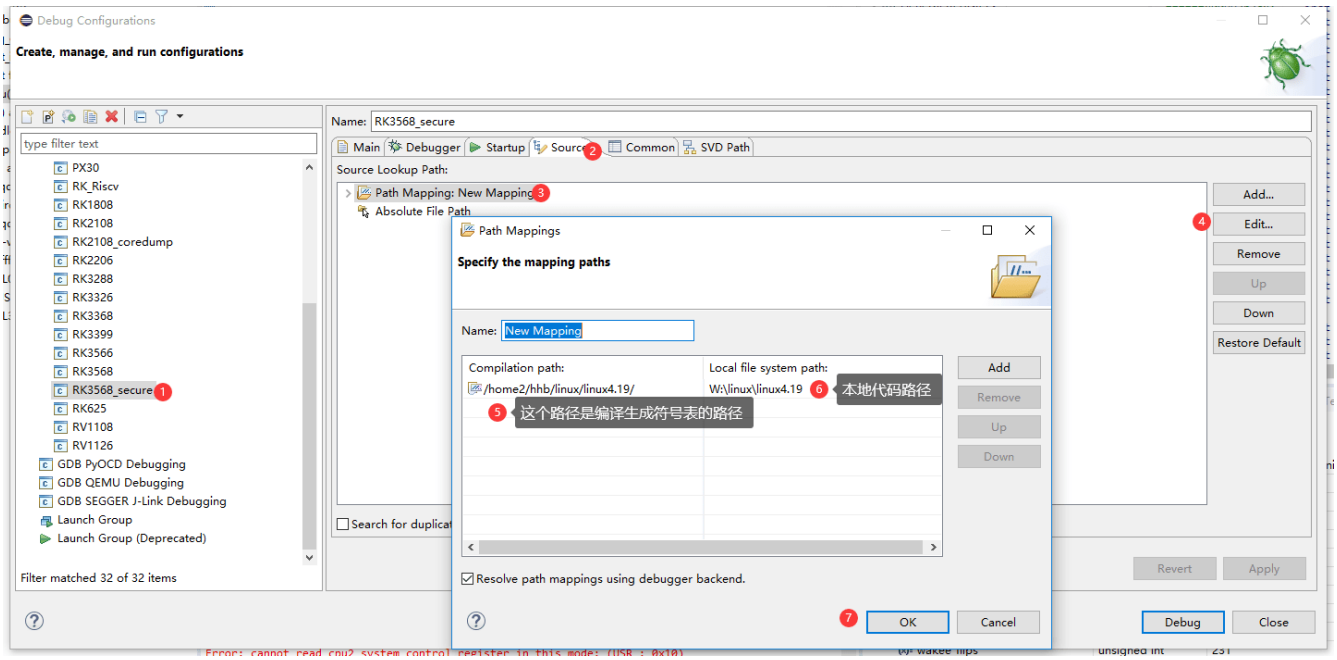


4.4 Configure Source Code Path

- Method 1: As shown in the figure below

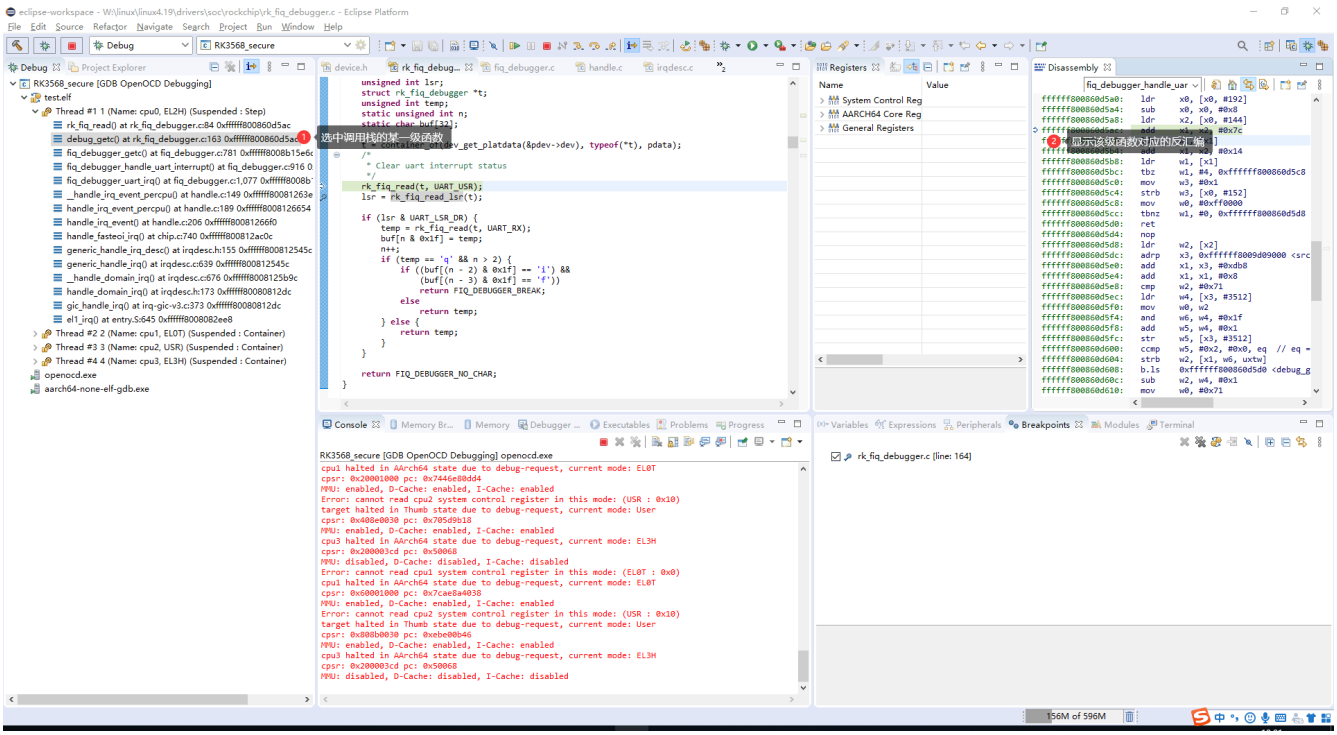


- Method 2: Configure settings in the Debug Configurations interface

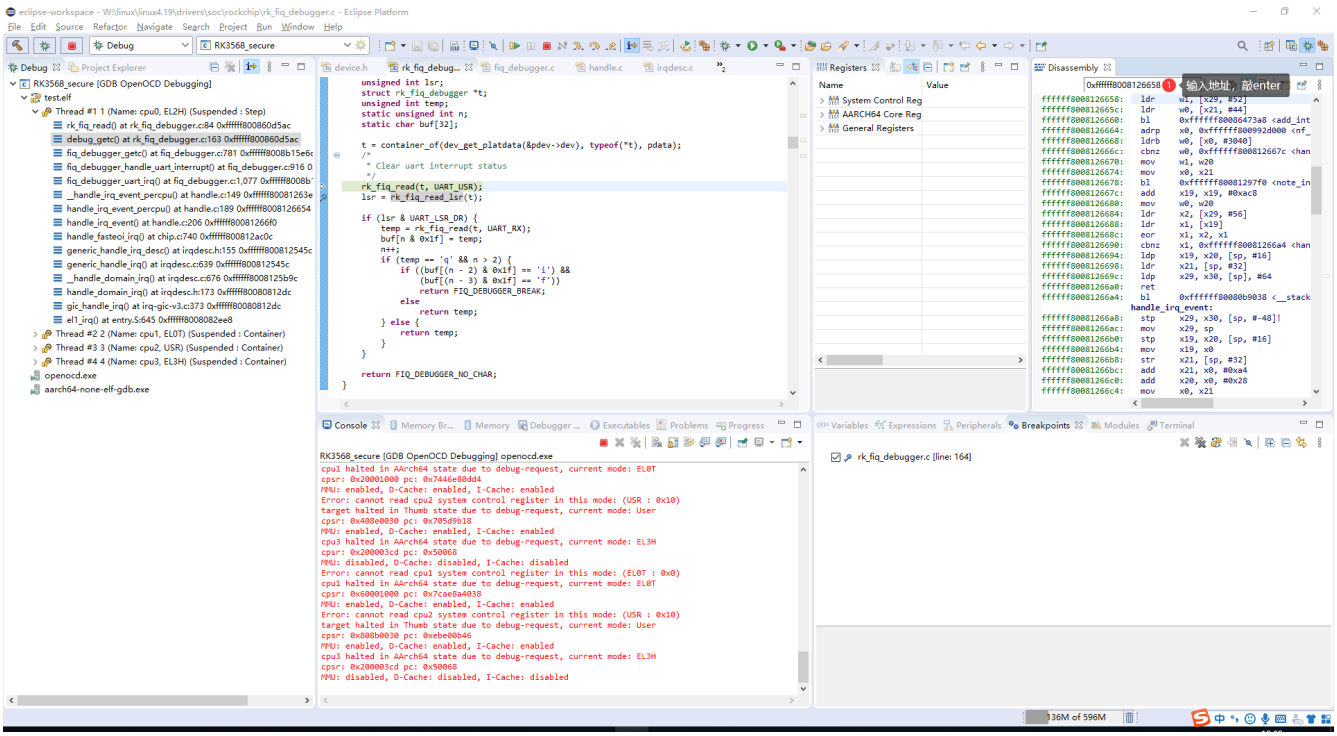


4.5 View Disassembly

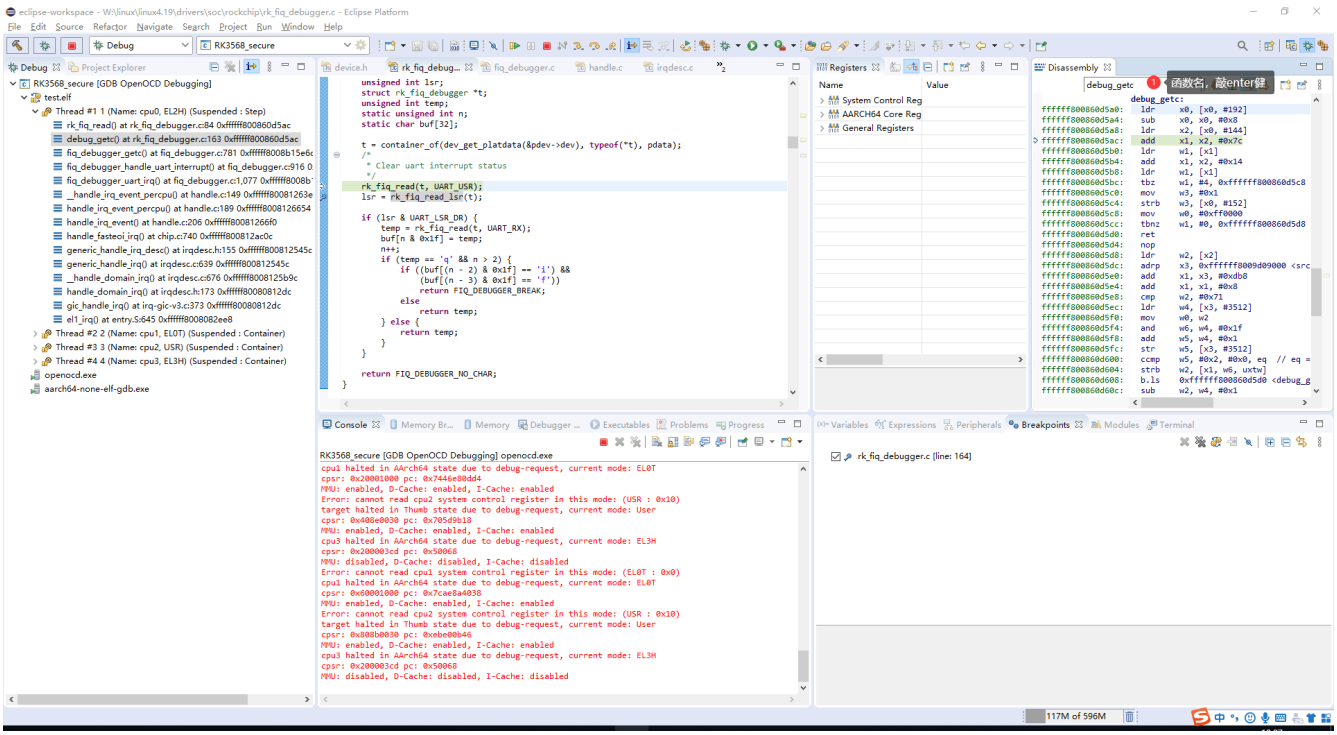
- View the disassembly of call stack functions



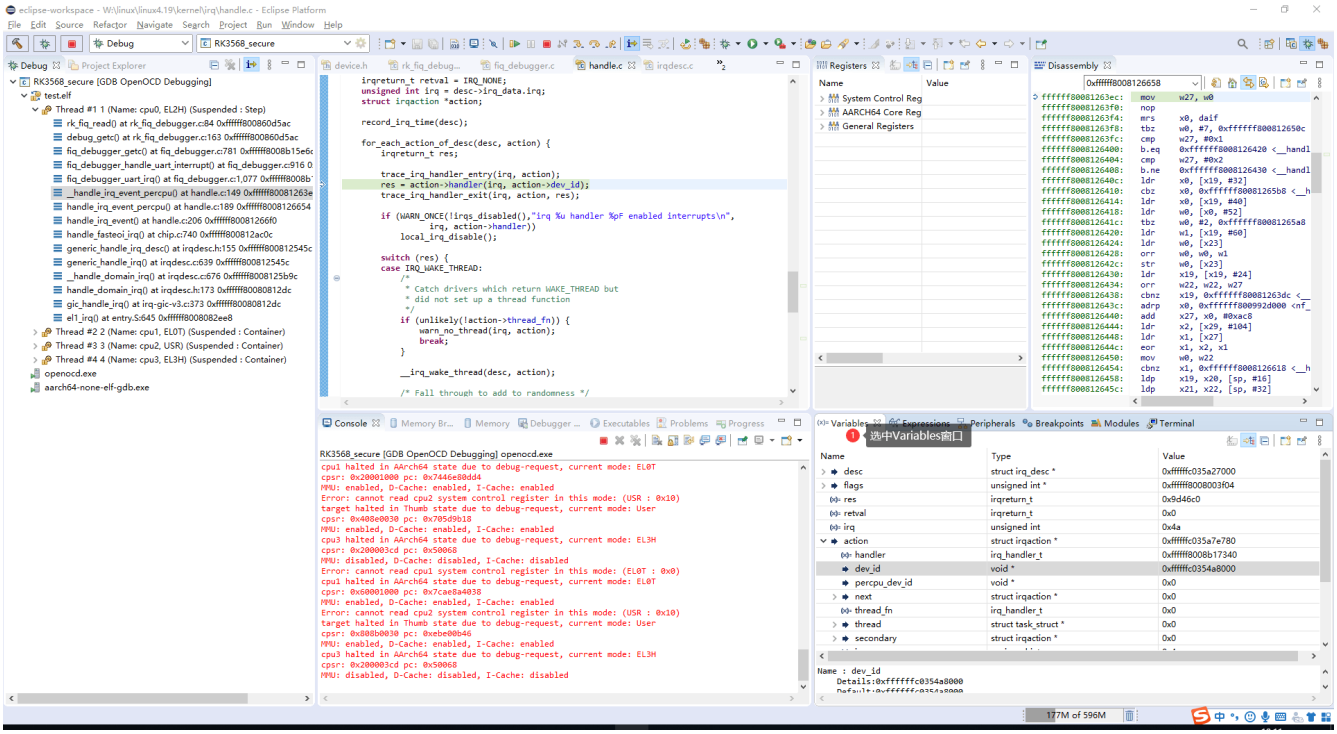
- Disassemble a specific address



To disassemble a specific function, a symbol table must be imported first.

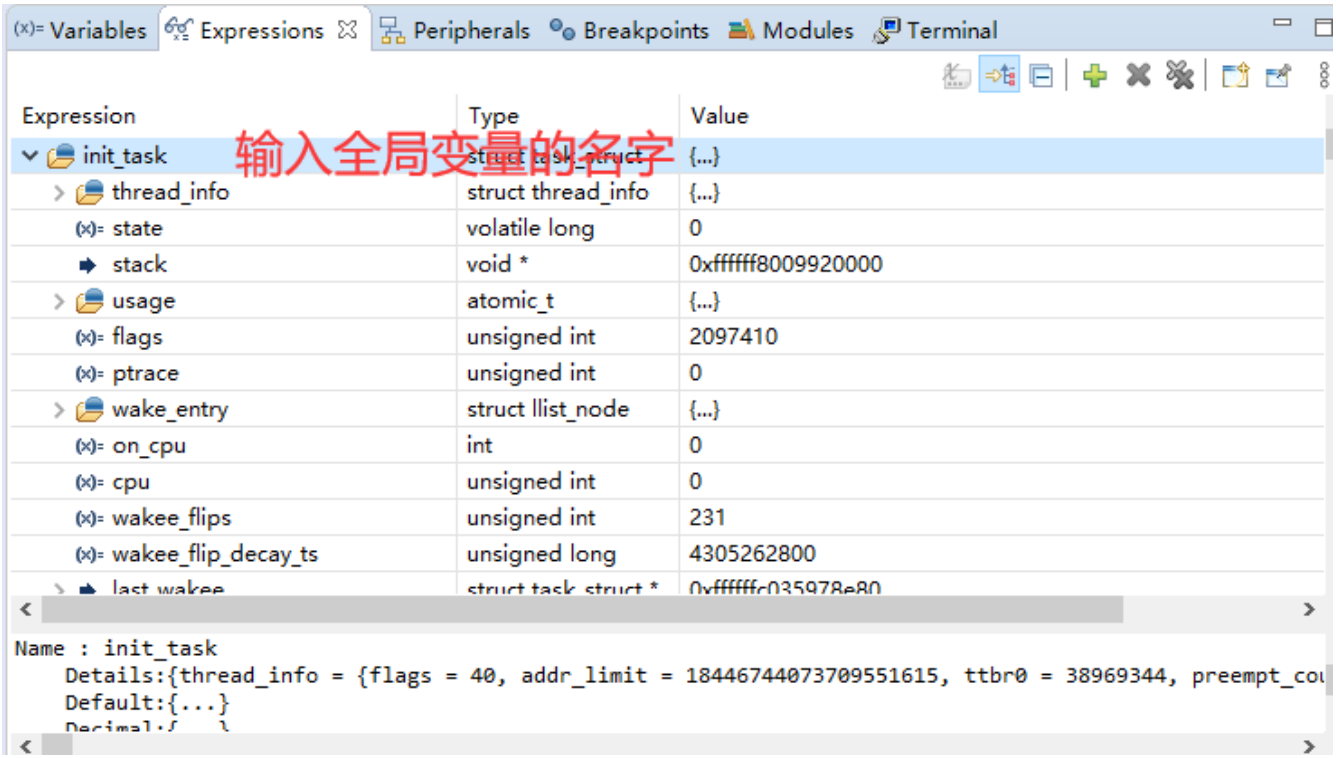


4.6 View Local Variables of Call Stack Functions

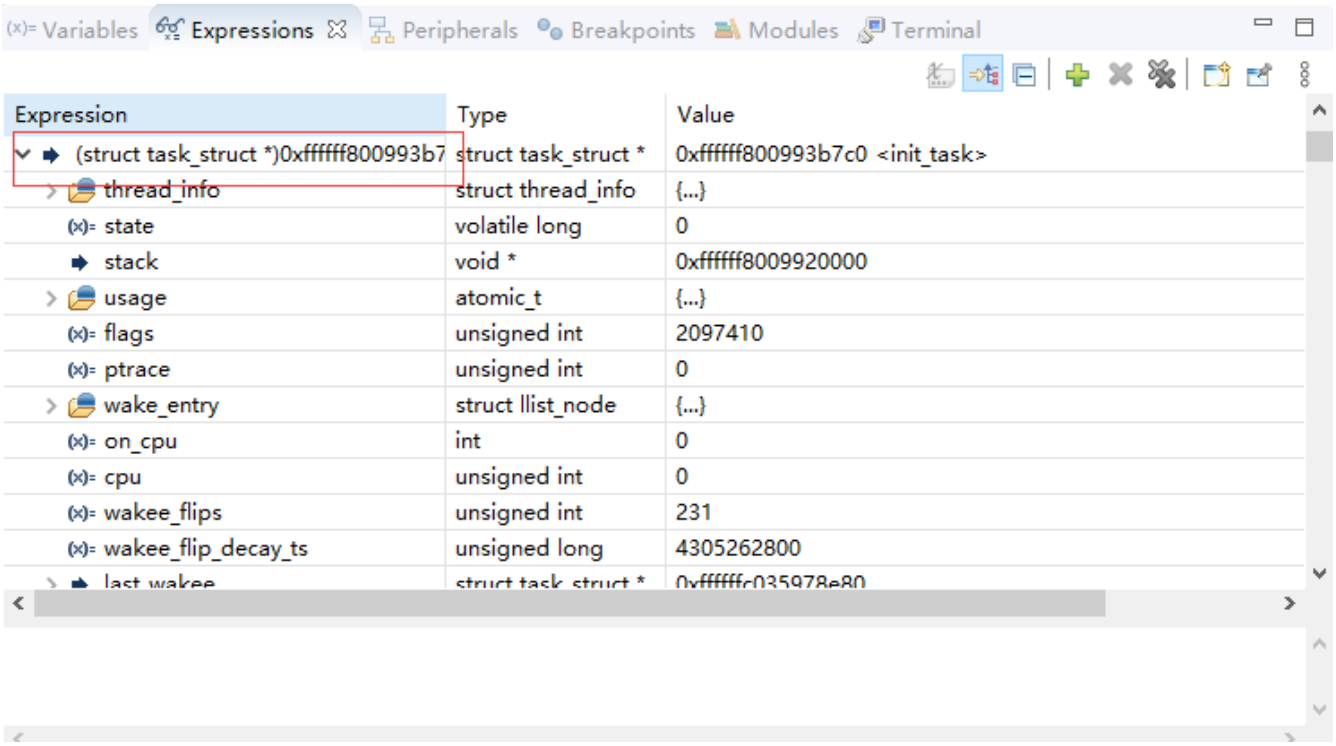


4.7 View Global Variables

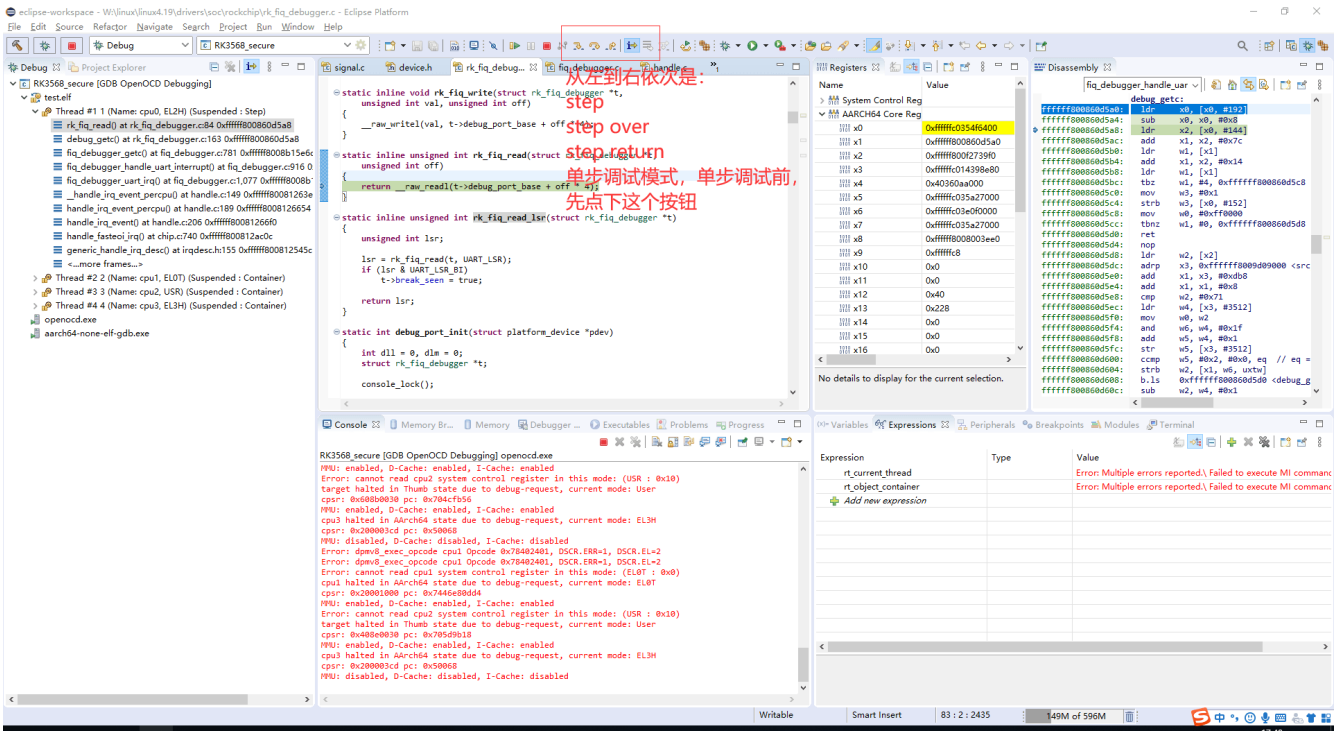
Click Add new expression in the Expressions window and enter the global variable name.



Click Add new expression in the Expressions window and enter the expression in pointer format.

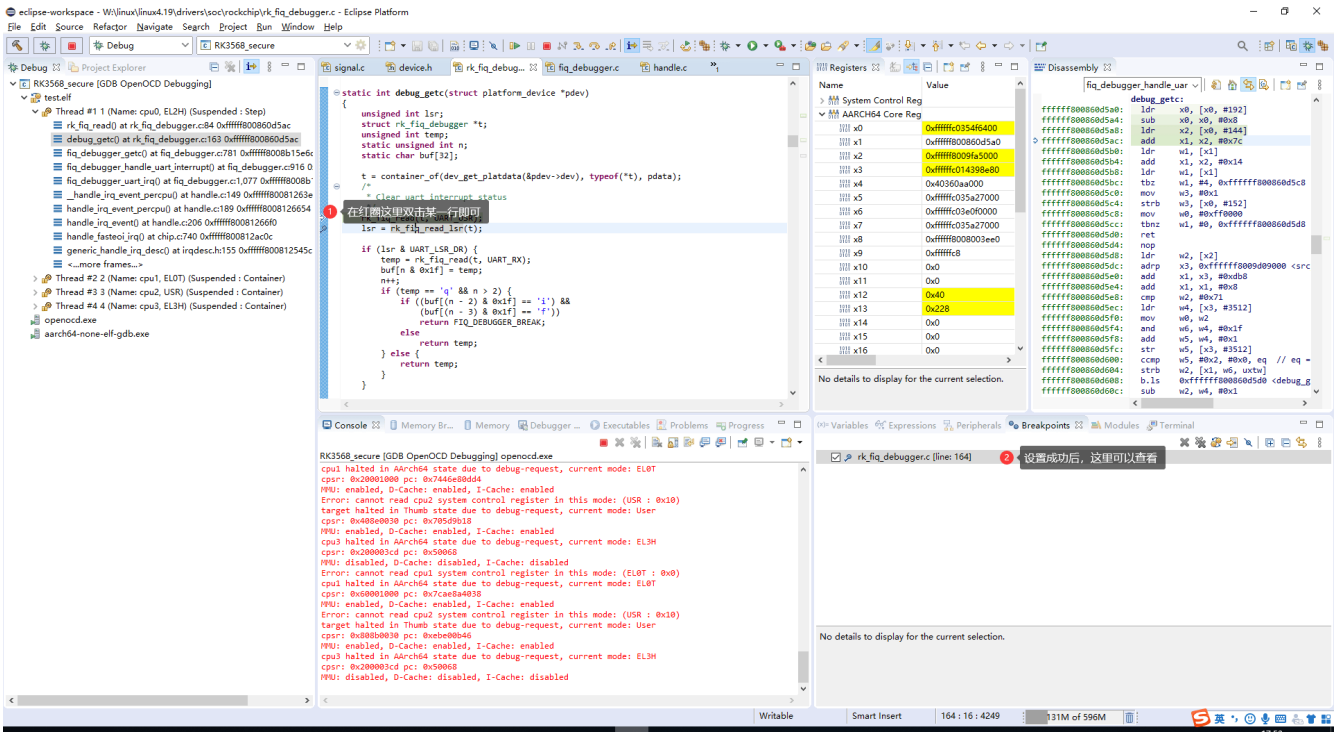


4.8 Single-Step Debugging

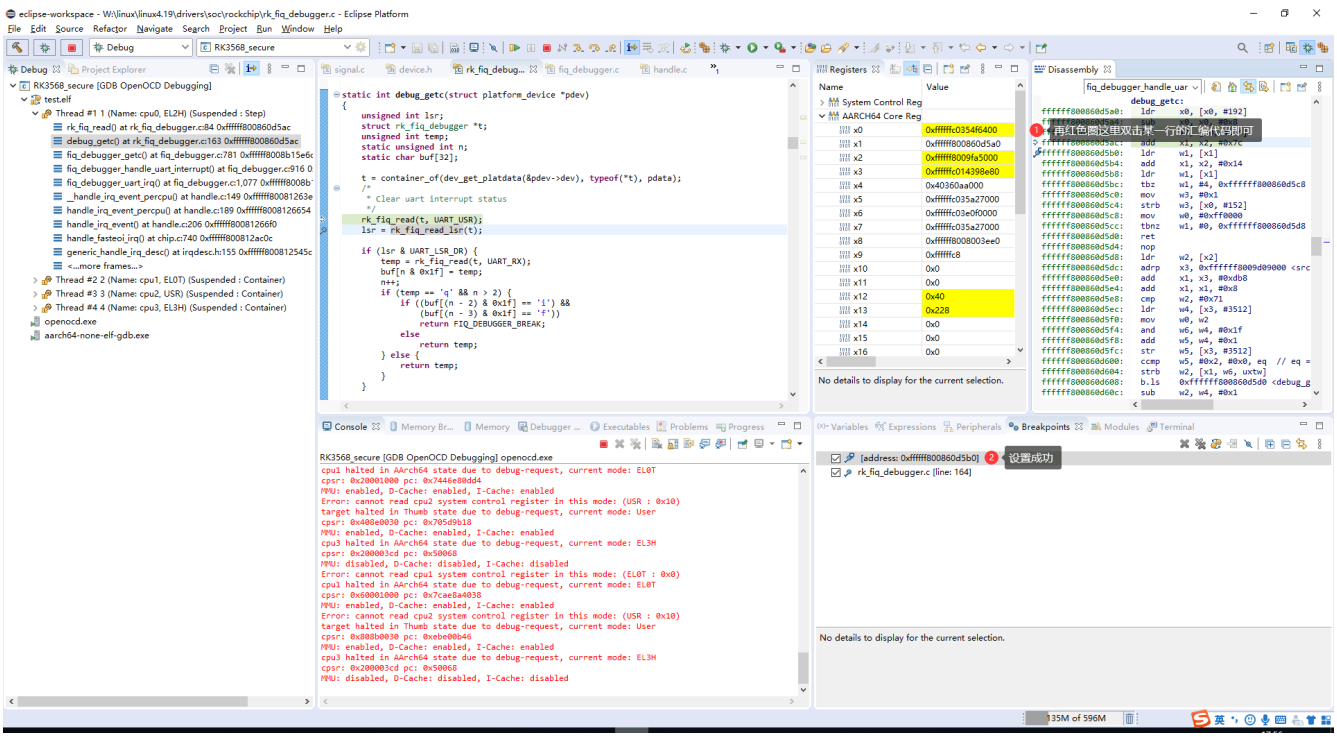


4.9 Setting Breakpoints

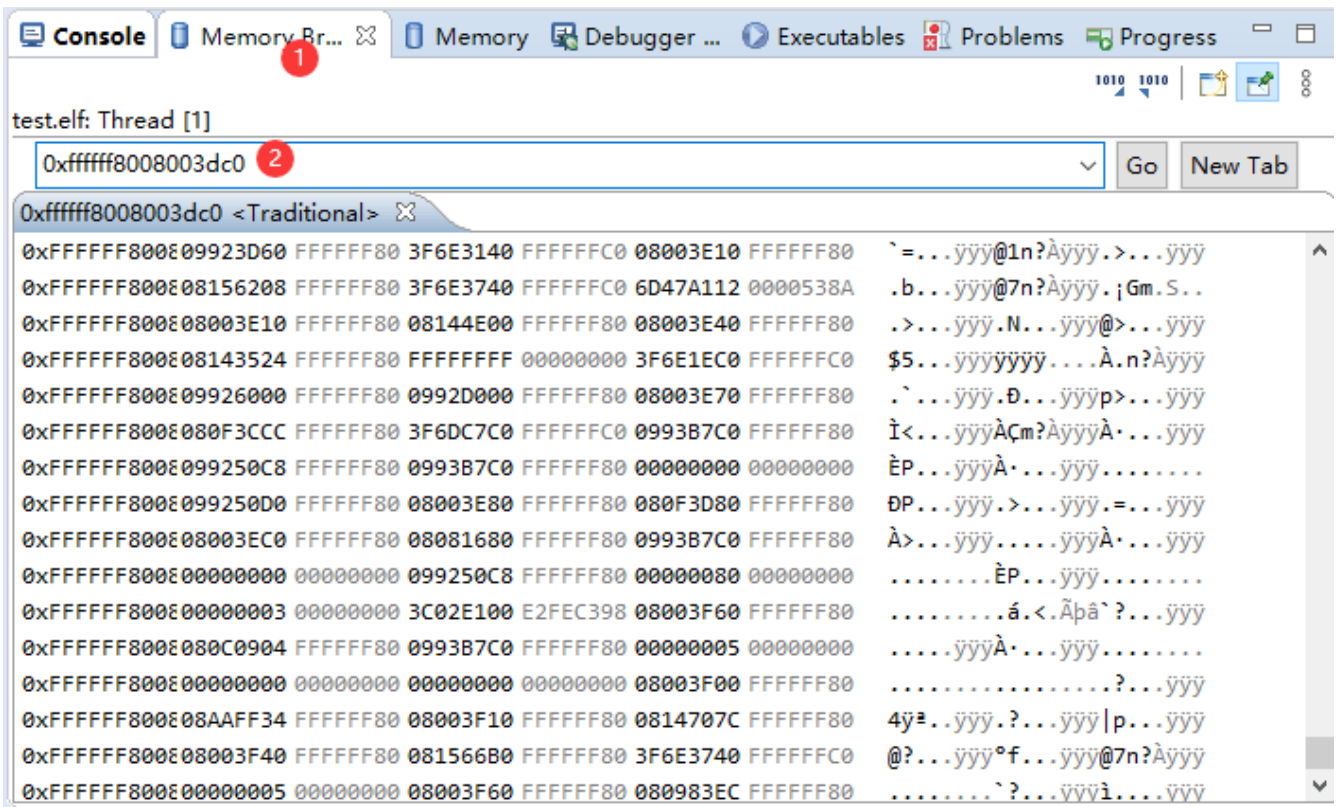
- Set breakpoints from the source code window



- Set breakpoints from the Disassembly window



4.10 View Memory Data

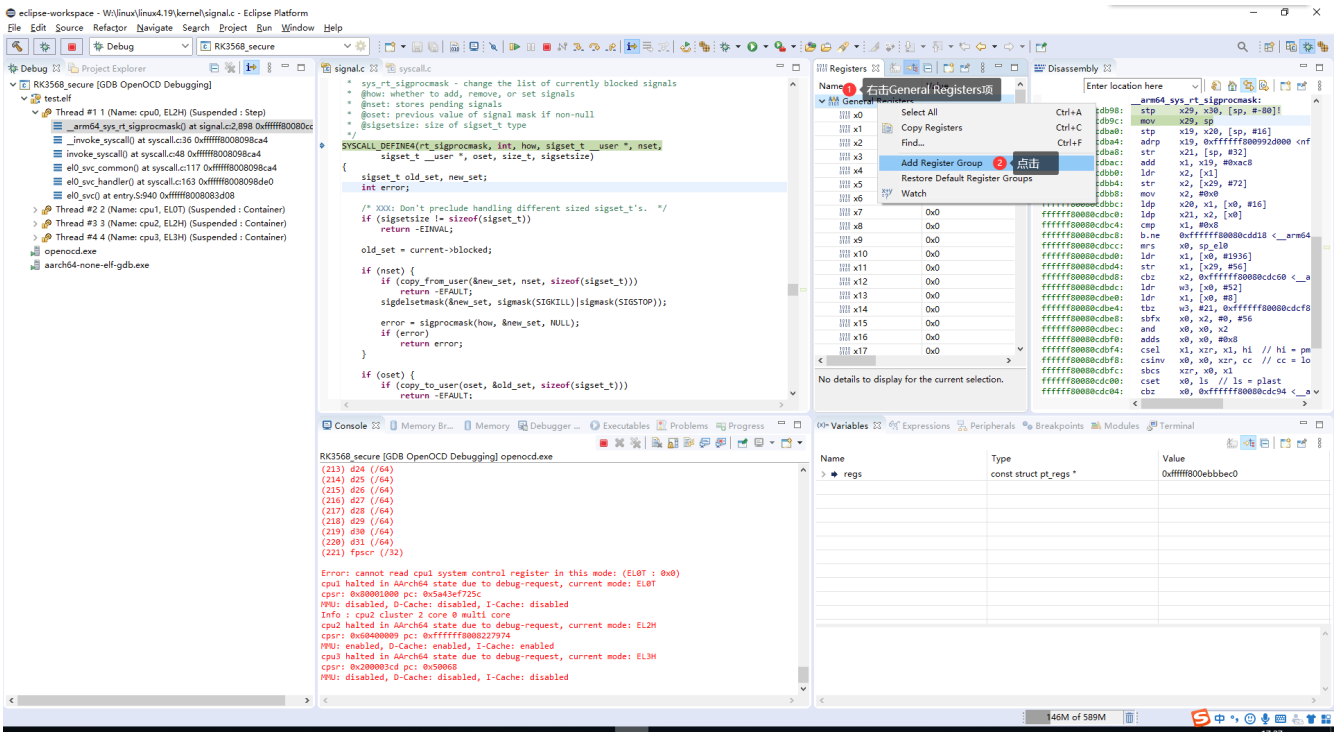


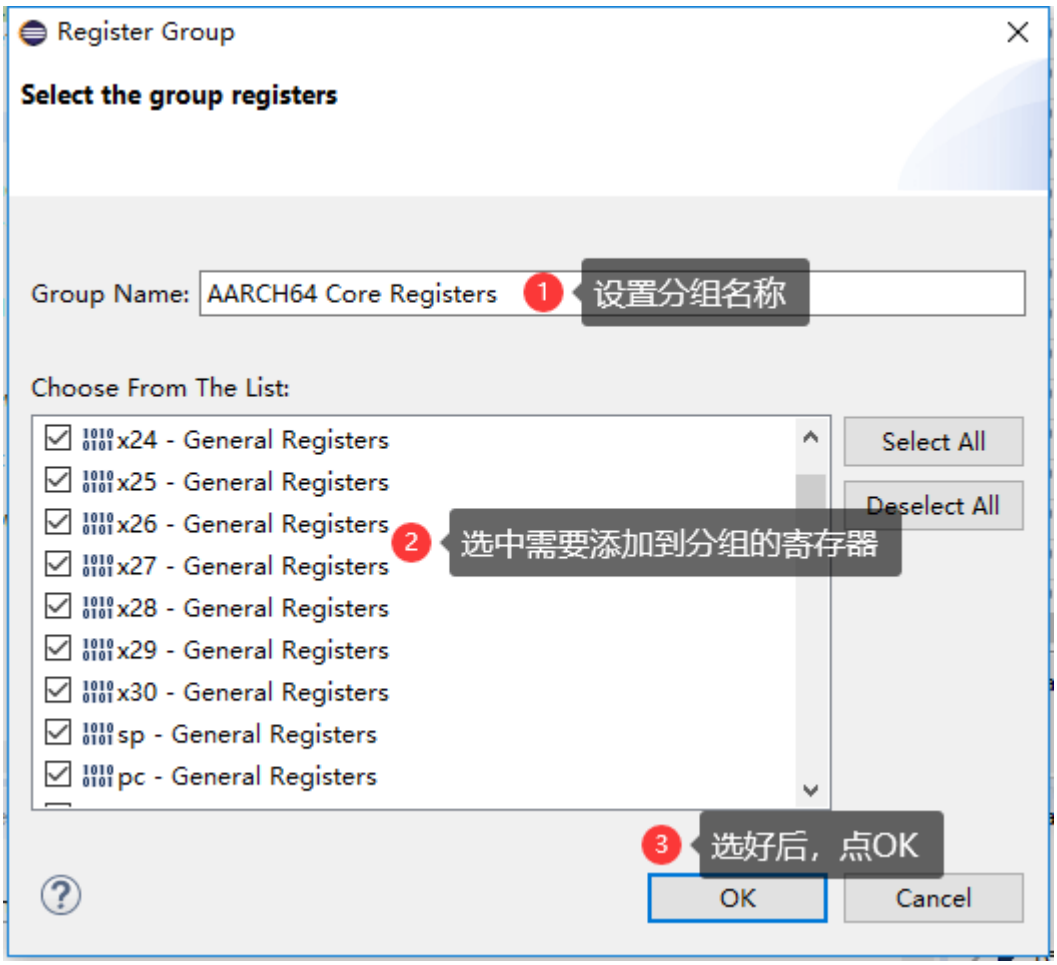
Note: The Memory Browser window only supports 32-bit address access correctly; 64-bit address access has issues. Please enter the OpenOCD command-line mode and use commands such as mdw, mww, smdw, smww, and io to operate memory.

5. Advanced Debugging Features

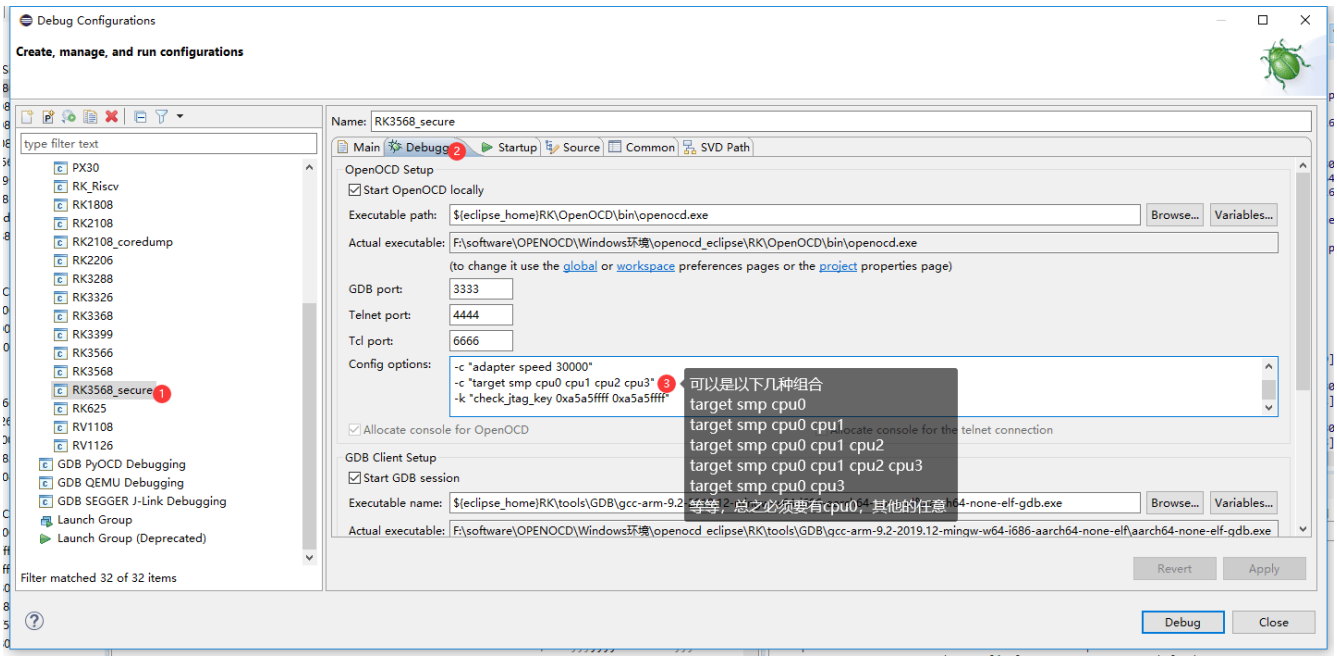
5.1 Register Groups

Due to the large number of Registers, viewing them individually is inconvenient; grouping them facilitates easier inspection.



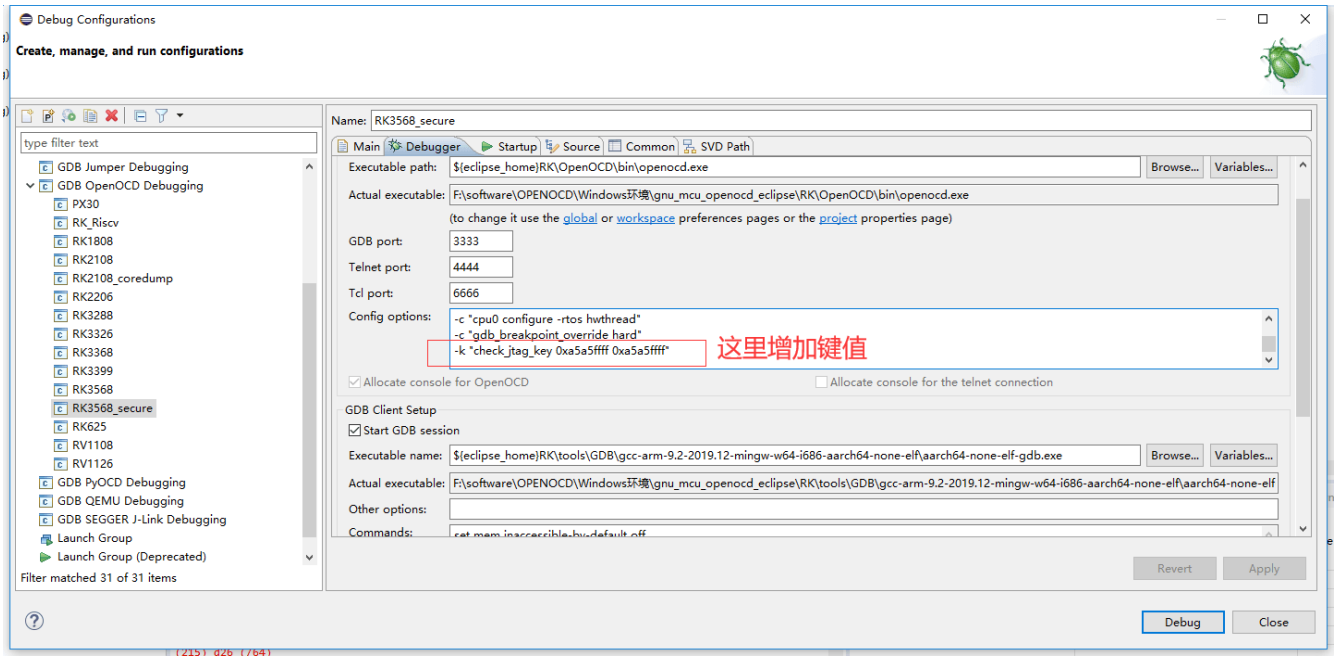


5.2 Specify the Connected CPU



5.3 Secure Debugging

If the customer product enables security policies, a key is required for JTAG debugging.



5.4 OpenOCD Command-Line Mode (Based on Command-Line Terminal)

Open OpenOCD in the cmd window to obtain usage help.



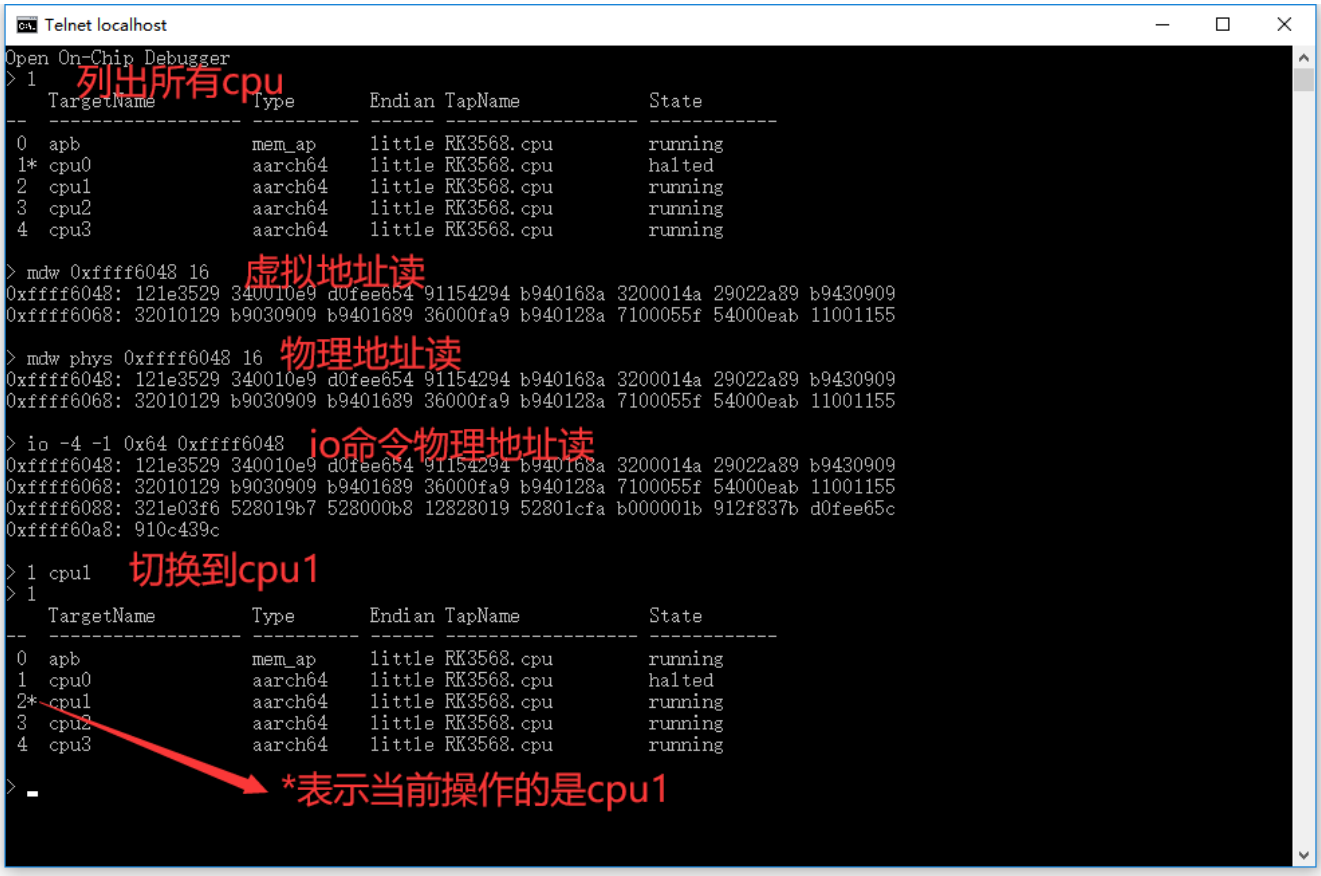
For more commands, refer to the OpenOCD User's Guide:

https://openocd.org/doc/html/General-Commands.html#General-Commands

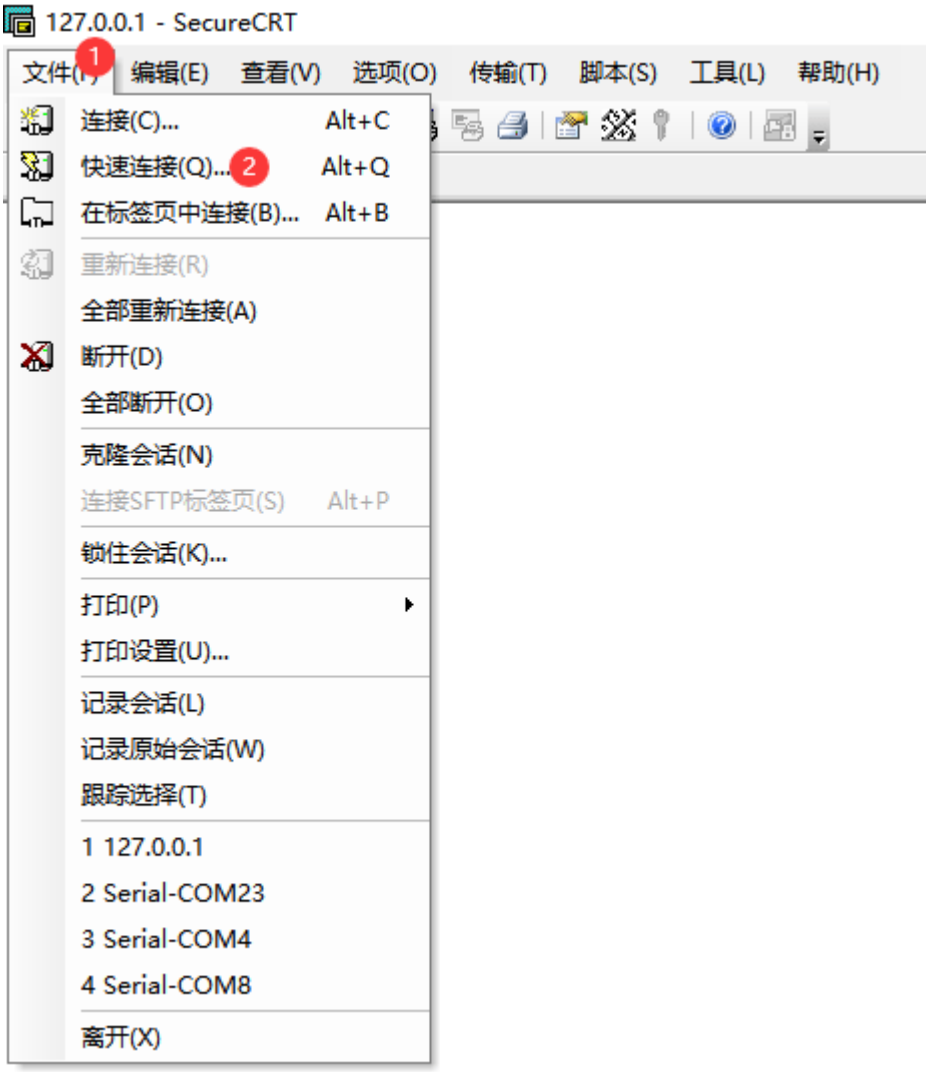
openocd.exe -r Connect to the chip

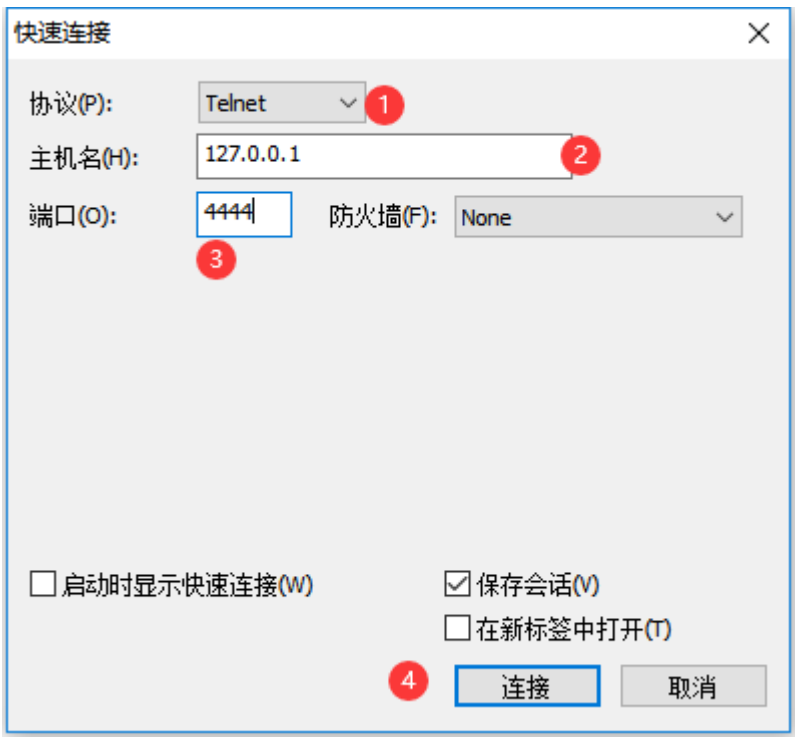


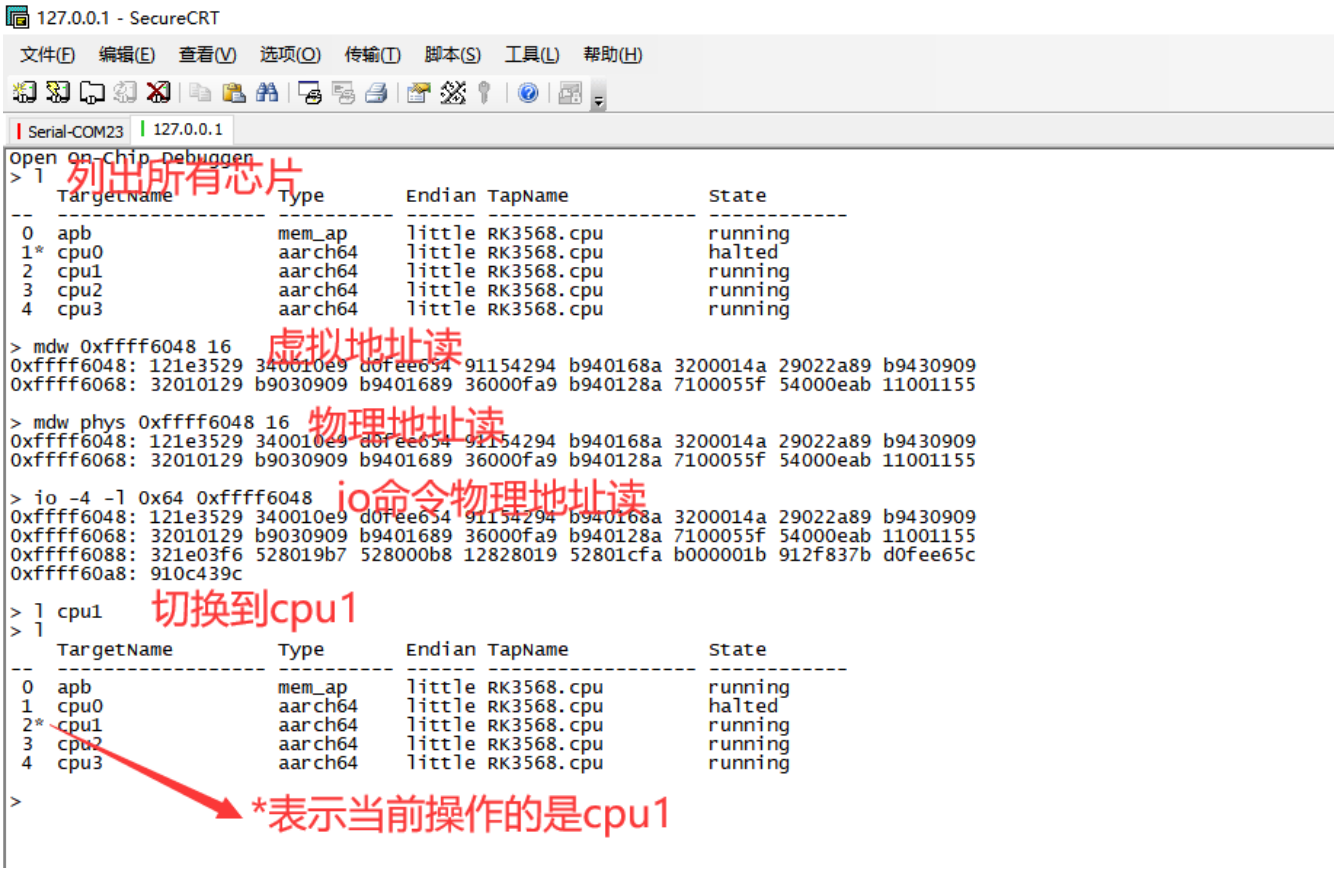
To establish a Telnet connection via the command prompt, open a new cmd window and execute: telnet localhost 4444



Create a Telnet connection based on SecureCRT


127.0.0.1 - SecureCRT
文件(F)
编辑(E)
查看(V)
选项(O)
传输(T)
脚本(S)
工具(L)
帮助(H)
1
连接(C)... Alt+C
快速连接(Q)... Alt+Q
2
在标签页中连接(B)... Alt+B
重新连接(R)
全部重新连接(A)
断开(D)
全部断开(O)
克隆会话(N)
连接SFTP标签页(S) Alt+P
锁住会话(K)...
打印(P)
打印设置(U)...
记录会话(L)
记录原始会话(W)
跟踪选择(T)
1 127.0.0.1
2 Serial-COM23
3 Serial-COM4
4 Serial-COM8
离开(X)

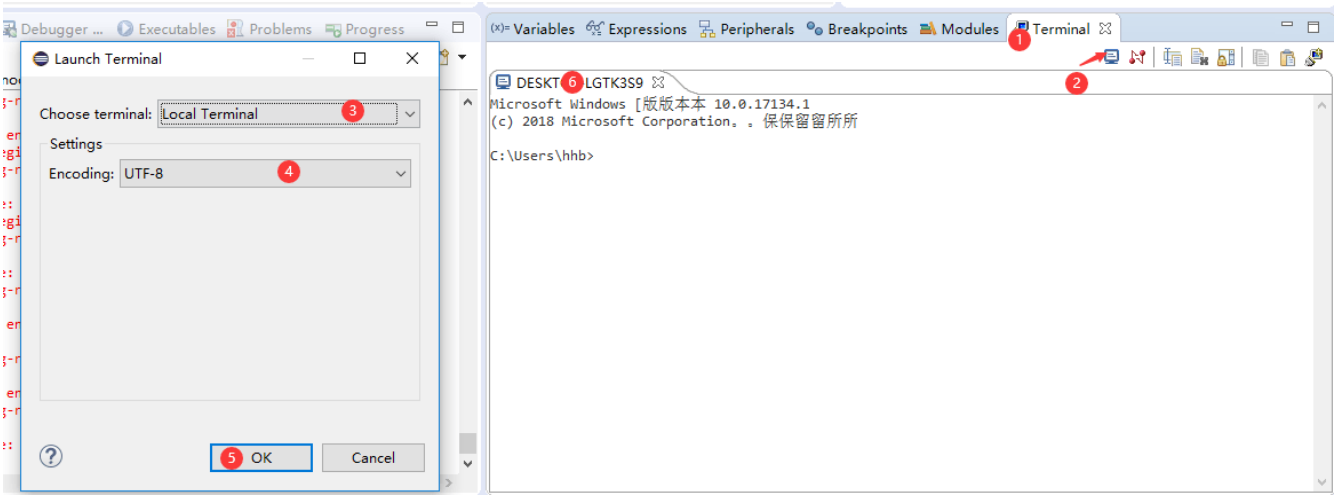

快速连接
协议(P):
Telnet
1
主机名(H):
127.0.0.1
2
端口(O):
4444
3
防火墙(F):
None
启动时显示快速连接(W)
保存会话(V)
在新标签中打开(T)
4
连接
取消



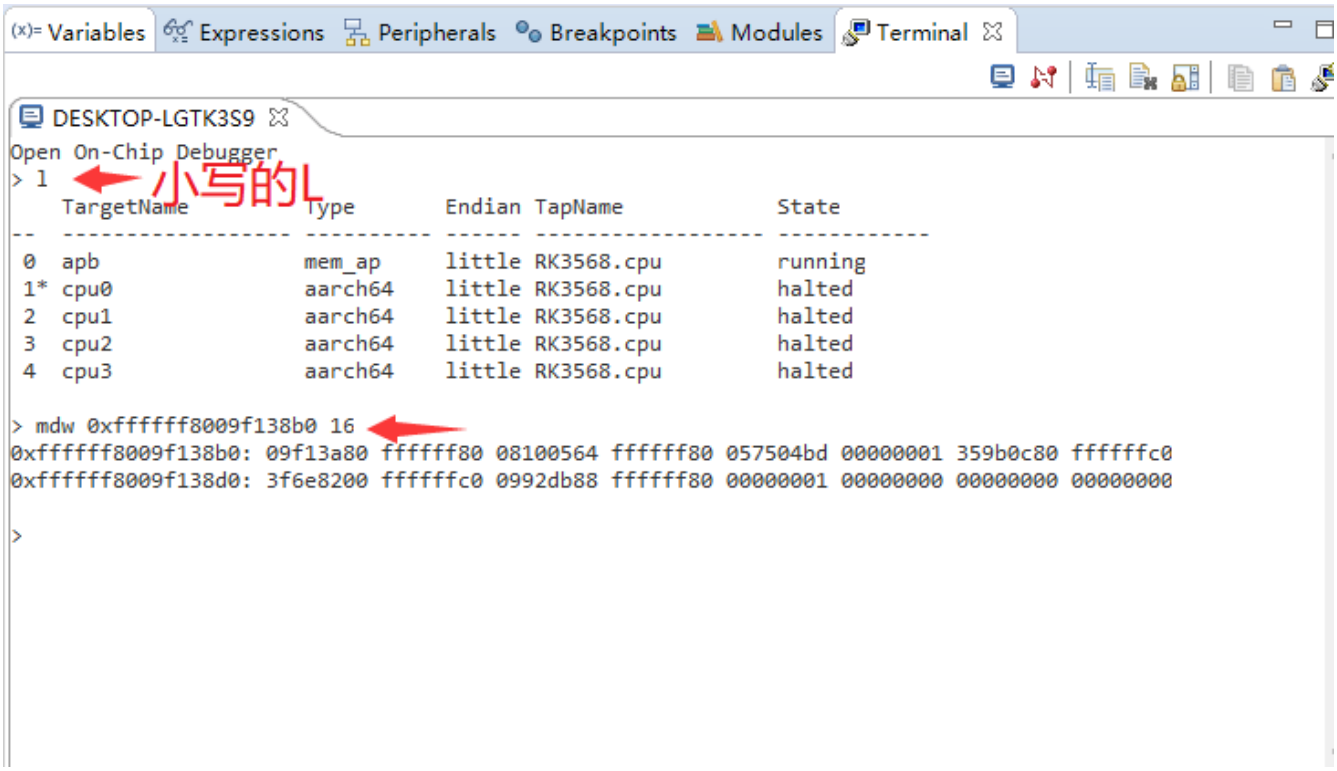
5.5 OpenOCD Command-Line Mode (Based on Eclipse)

In some cases, the UI cannot effectively display or operate certain information, necessitating the use of OpenOCD command-line mode.

Open a local terminal.

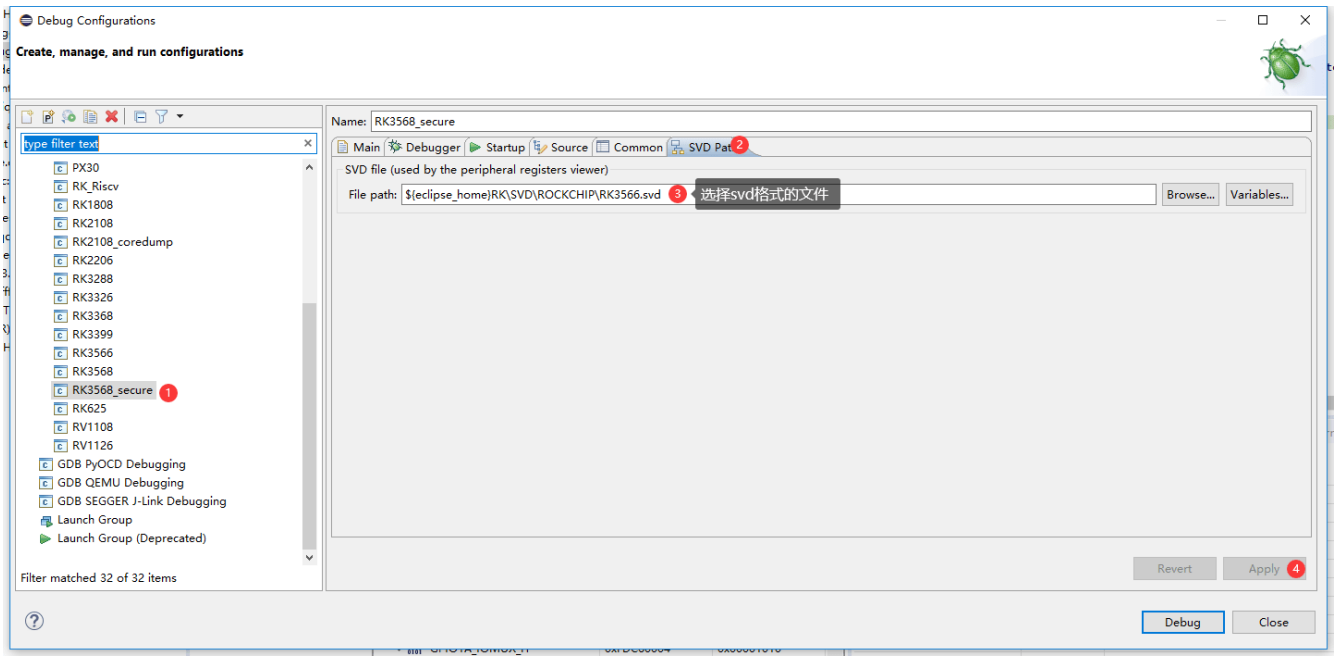


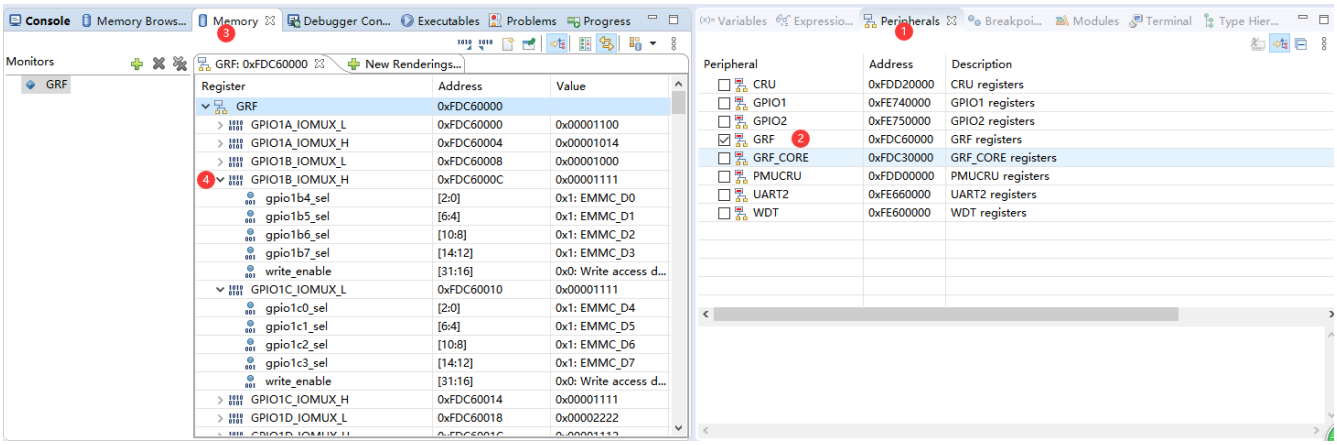
Enter command-line mode by running `telnet localhost 4444`. If the system reports that the `telnet` command is unrecognized, refer to the section "Enable Windows Telnet Feature".



5.6 View Visualized Peripheral Registers

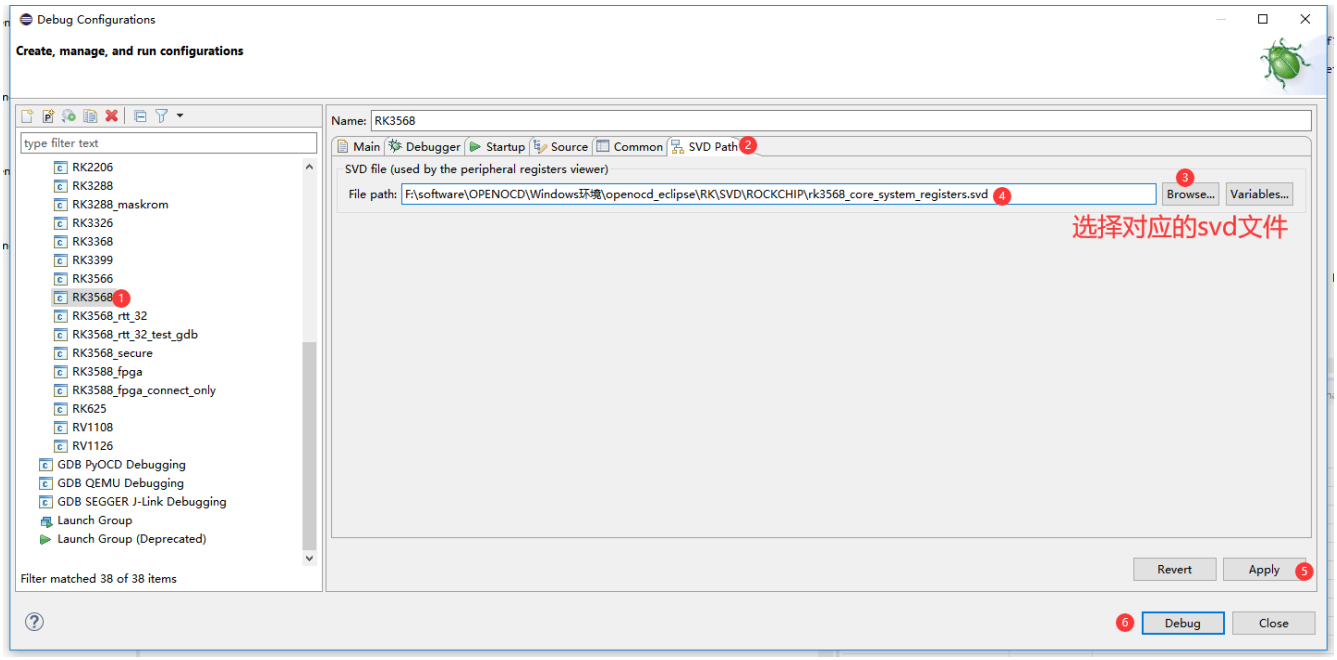
Before connecting, select the SVD format file.

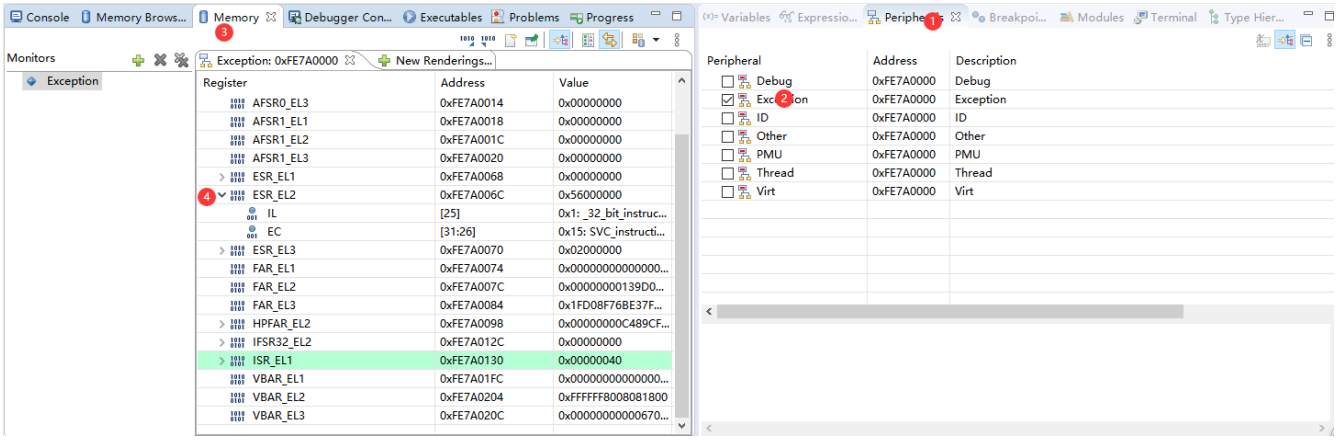




5.7 View Visualized System Control Registers

Before connecting, select the SVD format file.





5.8 Export Data from Memory to File

```
dump_image filename [p:]address size
Virtual address access:
> dump_image dd.bin 0xffffff8009a63c80 65536
dumped 65536 bytes in 0.113187s (565.436 KiB/s)
Physical Address Access:
> dump_image dd.bin p:0x01c63c80 65536
dumped 65536 bytes in 0.094566s (676.776 KiB/s)
```

5.9 Import Data from File to Memory

```
load_image filename [p:]address ['bin'|'ihex'|'elf'|'s19'] [min_address] [max_length]
Virtual address access:
> load_image dd.bin 0xffffff8009a63c80 bin
65536 bytes written at address 0xffffff8009a63c80
downloaded 65536 bytes in 0.055928s (1144.328 KiB/s)
Physical Address Access:
> load_image dd.bin p:0x01c63c80 bin
65536 bytes written at address 0x01c63c80
downloaded 65536 bytes in 0.056400s (1134.752 KiB/s)
```

5.10 Compare File Data with Memory Data

```
verify_image filename [offset [type]]
Only virtual address access is supported:
> verify_image dd.bin 0xffffff8009a63c80
verified 65536 bytes in 0.066775s (958.443 KiB/s)
```

Note: The above three commands involve both virtual and physical addresses. It is particularly important to note that for maskrom, tpl, spl, uboot, ATF, etc., virtual addresses can be considered identical to physical addresses; therefore, only the virtual address access method is required. However, for Linux, physical and virtual addresses differ. When accessing a physical address, prefix the address with `p:`.

Additionally, if the filename is a relative path, `dd.bin` will be saved in the directory where OpenOCD is executed.

6. Practical Application Scenarios

6.1 Debugging CPU Hang Issues

Symptoms such as a frozen display, black screen, unresponsive serial console, or relevant kernel errors indicate that the CPU has ceased executing instructions, likely hanging on one or more specific instructions.

Taking the RK3399 boot hang as an example, the kernel log shown below indicates a hang: the log output essentially stops, the serial console becomes unresponsive, and the display fails to update.

```
[    7.441882] type=1400 audit(1501952314.540:9): avc: denied { setattr } for pid=1
comm="init" name="mmcblk1p13" dev="tmpfs" ino=15288 scontext=u:r:init:s0
tcontext=u:object_r:block_device:s0 tclass=lnk_file permissive=1
[    7.455950] init: Service 'exec 2 (/system/bin/vdc)' (pid 252) exited with status 0
rk3399_all:/ $ [    7.589883] random: nonblocking pool is initialized
[    8.078474] init: Starting service 'bootanim'...
[    8.261358] read channel() error: -110
[   15.245727] BUG: spinlock lockup suspected on CPU#2, adbd/251
```

```
$ ./src/openocd.exe -r rk3399
Open On-Chip Debugger 0.10.0+dev-01555-g6ad367fa3-dirty (2021-08-04-17:44)
Licensed under GNU GPL v2
For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
Many logs omitted in between.
Info : clock speed 2000 kHz
Info : SWD DPIDR 0x5ba02477
Info : cpu0: hardware has 6 breakpoints, 4 watchpoints
Info : cpu1: hardware has 6 breakpoints, 4 watchpoints
Info : cpu2: hardware has 6 breakpoints, 4 watchpoints
Info : cpu3: hardware has 6 breakpoints, 4 watchpoints
Info : cpu4: hardware has 6 breakpoints, 4 watchpoints
Info : cpu5: hardware has 6 breakpoints, 4 watchpoints
Info : starting gdb server for cpu0 on 3333
Info : Listening on port 3333 for gdb connections
Info : starting gdb server for cpu1 on 3334
Info : Listening on port 3334 for gdb connections
Info : starting gdb server for cpu2 on 3335
Info : Listening on port 3335 for gdb connections
Info : starting gdb server for cpu3 on 3336
Info : Listening on port 3336 for gdb connections
Info : starting gdb server for cpu4 on 3337
Info : Listening on port 3337 for gdb connections
Info : starting gdb server for cpu5 on 3338
Info : Listening on port 3338 for gdb connections
```

```
Open another command prompt window and enter telnet localhost 4444 to access the command-line
debugger.

Open On-Chip Debugger
> cpu_block    Check which commands are available for diagnosing CPU hang issues.
l View CPU status
dump_pc   Print the PC pointer multiple times
halt   Halt the CPU
if halt fails, try fhalt    Force the CPU into debug state
if fhalt succeeds, try core_reg64 or core_reg32, but you can't access memory
> l
    TargetName         Type       Endian TapName            State
```

```
--  ------------------ ---------- ------ ------------------ ------------
 0  ahb                mem_ap     little RK3399.cpu         running
 1  apb                mem_ap     little RK3399.cpu         running
 2* cpu0               aarch64    little RK3399.cpu         running
 3  cpu1               aarch64    little RK3399.cpu         running
 4  cpu2               aarch64    little RK3399.cpu         running
 5  cpu3               aarch64    little RK3399.cpu         running
 6  cpu4               aarch64    little RK3399.cpu         running
 7  cpu5               aarch64    little RK3399.cpu         running
> dump_pc
1
cpu0 pc:0xffffff8008143a88
cpu1 pc:0xffffff8008143a5c
cpu2 pc:0xffffff8008081078
cpu3 pc:0xffffff8008081078
cpu4 pc:0xffffff80081133c8
cpu5 pc:0xffffff800808248c
2
cpu0 pc:0xffffff8008143a88
cpu1 pc:0xffffff8008143a5c
cpu2 pc:0xffffff8008081078
cpu3 pc:0xffffff8008081078
cpu4 pc:0xffffff80081133c8
cpu5 pc:0xffffff800808248c
3
cpu0 pc:0xffffff8008143a88
cpu1 pc:0xffffff8008143a5c
cpu2 pc:0xffffff8008081078
cpu3 pc:0xffffff8008081078
cpu4 pc:0xffffff80081133c8
cpu5 pc:0xffffff800808248c
4
cpu0 pc:0xffffff8008143a88
cpu1 pc:0xffffff8008143a5c
cpu2 pc:0xffffff8008081078
cpu3 pc:0xffffff8008081078
cpu4 pc:0xffffff80081133c8
cpu5 pc:0xffffff800808248c
5
cpu0 pc:0xffffff8008143a88
cpu1 pc:0xffffff8008143a5c
cpu2 pc:0xffffff8008081078
cpu3 pc:0xffffff8008081078
cpu4 pc:0xffffff80081133c8
cpu5 pc:0xffffff800808248c    After five printouts, the PC pointers of all six CPUs remain
unchanged, indicating a likely system hang.

> halt
Timeout waiting for target cpu0 halt   First attempt to halt cpu0 failed

> fhalt
cpu0 fail to force entry to debug state; prsr:00000001  The second attempt to halt cpu0
failed; at this point, troubleshooting must rely on the printed PC pointer.
```

```
> l cpu1

> halt
Timeout waiting for target cpu1 halt

> fhalt
cpu1 fail to force entry to debug state; prsr:00000001
> l cpu2
> halt
Timeout waiting for target cpu2 halt

> fhalt
cpu2 successfully forced entry into debug state; prsr:00000011   CPU2 can now be halted

> core_reg64     CPU general-purpose registers can be printed for further analysis.
x0 (/64): 0xffffffc00a3d7e00
x1 (/64): 0xffffff8008081028
x2 (/64): 0x0000000000000000
x3 (/64): 0x00000040edbb6000
x4 (/64): 0x0100000000000000
x5 (/64): 0x000000000d8031bf
x6 (/64): 0x00000000000126a
x7 (/64): 0xffffff8008141ee0
x8 (/64): 0x0000000000000000
x9 (/64): 0x0000000000000000
x10 (/64): 0x00000000000012b0
x11 (/64): 0x000000000000000f
x12 (/64): 0x0000000100000000
x13 (/64): 0x0000000000000001
x14 (/64): 0x0000000000000000
x15 (/64): 0x0000000000000000
x16 (/64): 0xffffff80080b8b28
x17 (/64): 0x0000007cf3aeee00
x18 (/64): 0x0000000000000000
x19 (/64): 0x000000000000001e
x20 (/64): 0x0000008000000000
x21 (/64): 0xffffff80094e7990
x22 (/64): 0xffffff8009357ea0
x23 (/64): 0xffffffc00a3d7e00
x24 (/64): 0x0000000000000001
x25 (/64): 0xffffffc0f6ed4060
x26 (/64): 0xffffffc0f6ed8050
x27 (/64): 0x0000000000ed9234
x28 (/64): 0xffffffc00a3cad00
x29 (/64): 0xffffffc0f6ed8000
x30 (/64): 0xffffff8008083230
sp (/64): 0xffffffc0f6ed8000
pc (/64): 0xffffff800808107c

> mrs SCTLR_EL1   The system control register can also be read.
Cannot reach EL 0, SPSR corrupted?
SCTLR_EL1:0x34d5d91d
```

```
> mrs SCTLR_EL2
Cannot reach EL 0, SPSR corrupted?
SCTLR_EL2:0x30c50830

> mrs SCTLR_EL3
Cannot reach EL 0, SPSR corrupted?
SCTLR_EL3:0x00cd383f
```

```
Execute `objdump.exe -d vmlinux | less` to disassemble and view the function and instruction
where the PC pointer is located.

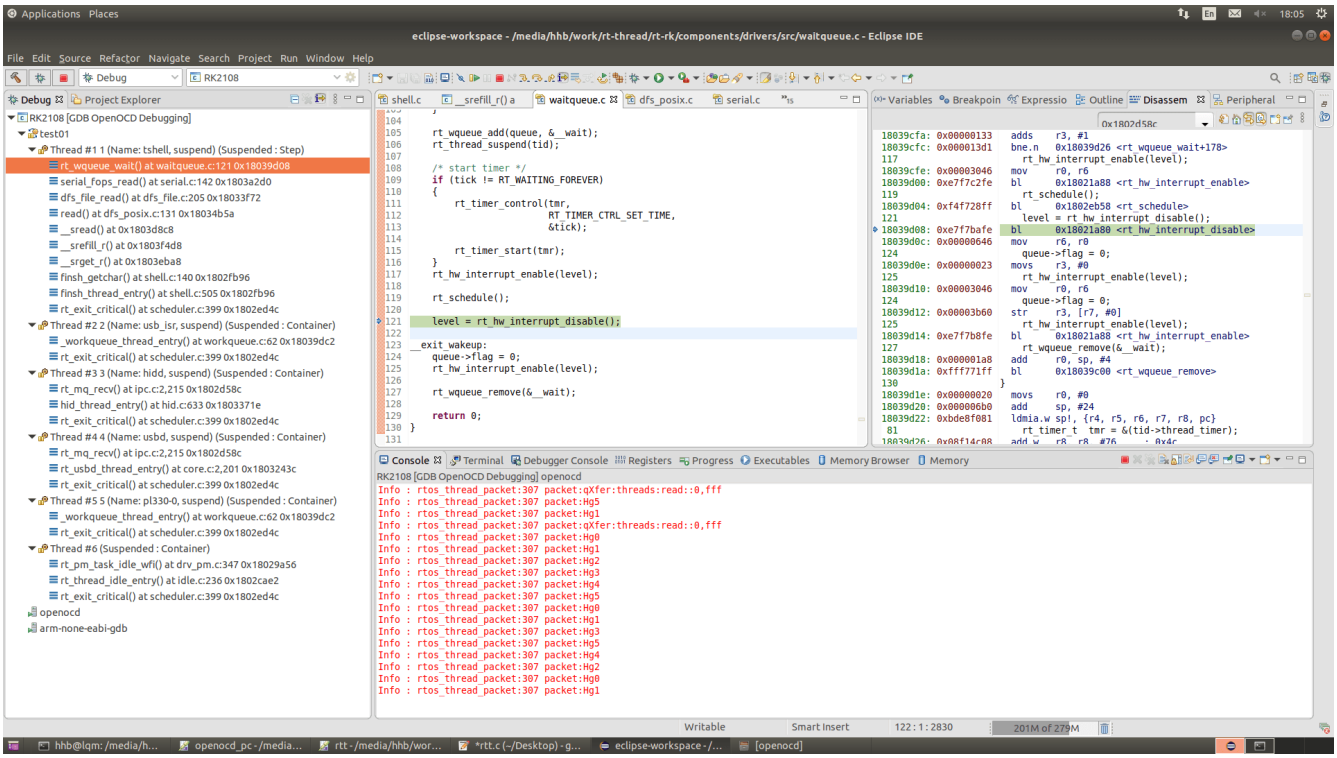
ffffff8008081028 <gic_handle_irq>:
ffffff8008081028:       a9bc7bfd        stp     x29, x30, [sp, #-64]!
ffffff800808102c:       910003fd        mov     x29, sp
ffffff8008081030:       a9025bf5        stp     x21, x22, [sp, #32]
ffffff8008081034:       d00096b6        adrp    x22, ffffff8009357000 <event_hash+0x288>
ffffff8008081038:       a90363f7        stp     x23, x24, [sp, #48]
ffffff800808103c:       d000a335        adrp    x21, ffffff80094e7000
<event_class_drv_config+0x40>
ffffff8008081040:       aa0003f7        mov     x23, x0
ffffff8008081044:       913a82d6        add     x22, x22, #0xea0
ffffff8008081048:       912642b5        add     x21, x21, #0x990
ffffff800808104c:       a90153f3        stp     x19, x20, [sp, #16]
ffffff8008081050:       52800038        mov     w24, #0x1                       // #1
ffffff8008081054:       d503201f        nop
ffffff8008081058:       14000008        b       ffffff8008081078 <gic_handle_irq+0x50>
ffffff800808105c:       71003e7f        cmp     w19, #0xf
ffffff8008081060:       540007e8        b.hi    ffffff800808115c <gic_handle_irq+0x134>  //
b.pmore
ffffff8008081064:       92407e60        and     x0, x19, #0xffffffff
ffffff8008081068:       d518cc20        msr     s3_0_c12_c12_1, x0
ffffff800808106c:       d5033fdf        isb
ffffff8008081070:       d503201f        nop
ffffff8008081074:       14000035        b       ffffff8008081148 <gic_handle_irq+0x120>
ffffff8008081078:       d538cc13        mrs     x19, s3_0_c12_c12_0
*ffffff800808107c:       d5033f9f        dsb     sy     CPU2 hangs at this location
ffffff8008081080:       51004260        sub     w0, w19, #0x10
ffffff8008081084:       2a1303f4        mov     w20, w19
ffffff8008081088:       710fac1f        cmp     w0, #0x3eb
ffffff800808108c:       54000089        b.ls    ffffff800808109c <gic_handle_irq+0x74>  //
b.plast
ffffff8008081090:       5283ffe0        mov     w0, #0x1fff                     // #8191
ffffff8008081094:       6b00027f        cmp     w19, w0
ffffff8008081098:       54fffe29        b.ls    ffffff800808105c <gic_handle_irq+0x34>  //
b.plast
```

If the customer is still unable to analyze the cause of the system hang after completing the above steps, they can send the logs generated by the aforementioned commands to Rockchip engineers for analysis.

6.2 RT-Thread Debugging Instructions

By default, only a single thread is displayed. After adding the following commands, multiple threads can be displayed.

```
-c "cpu0 configure -rtos RT_Thread"
```



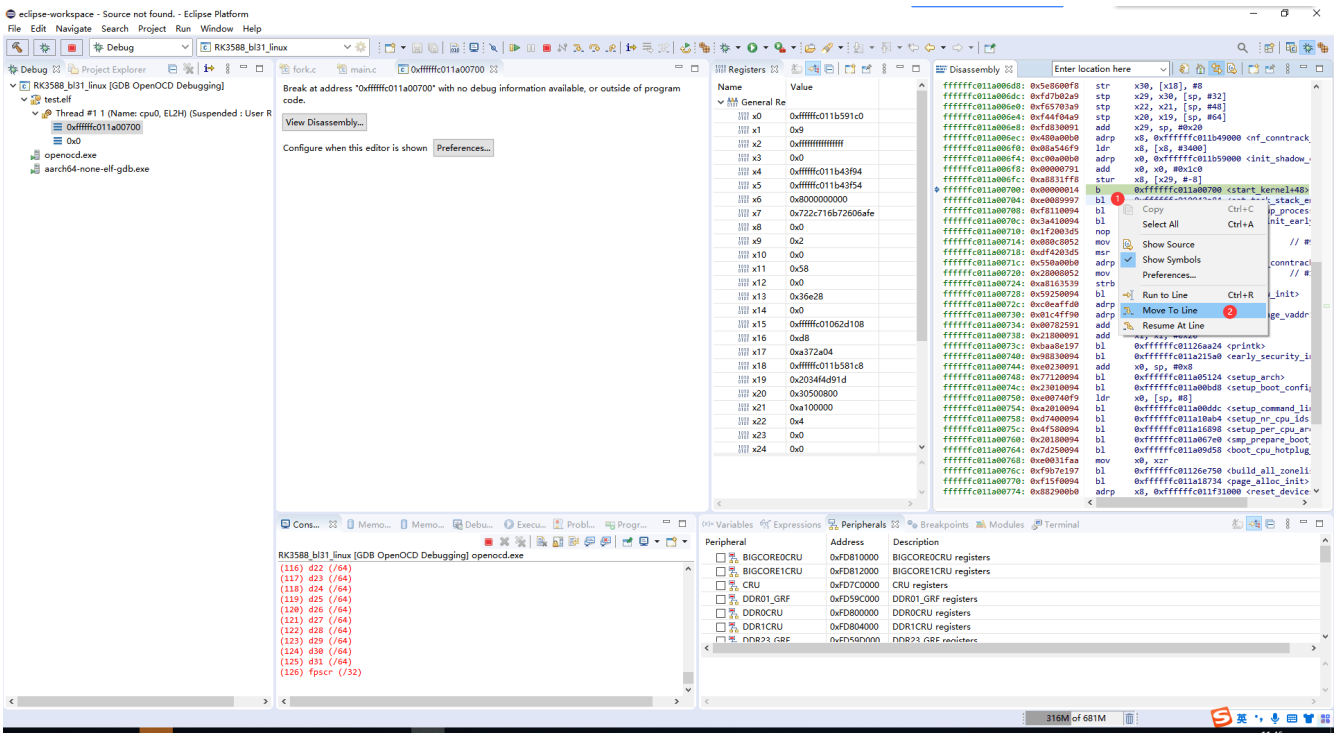
6.3 Single-Step Debugging of Linux

If no kernel logs are output during the kernel boot process, and serial port issues have been ruled out, it is highly likely that an exception occurred early in the kernel boot sequence.

6.3.1 Single-Core Kernel Phase

```
linux5.10/init/main.c
asmlinkage __visible void __init __no_sanitize_address start_kernel(void)
{
    char *command_line;
    char *after_dashes;
    asm volatile("b .");
 ...
 }
```

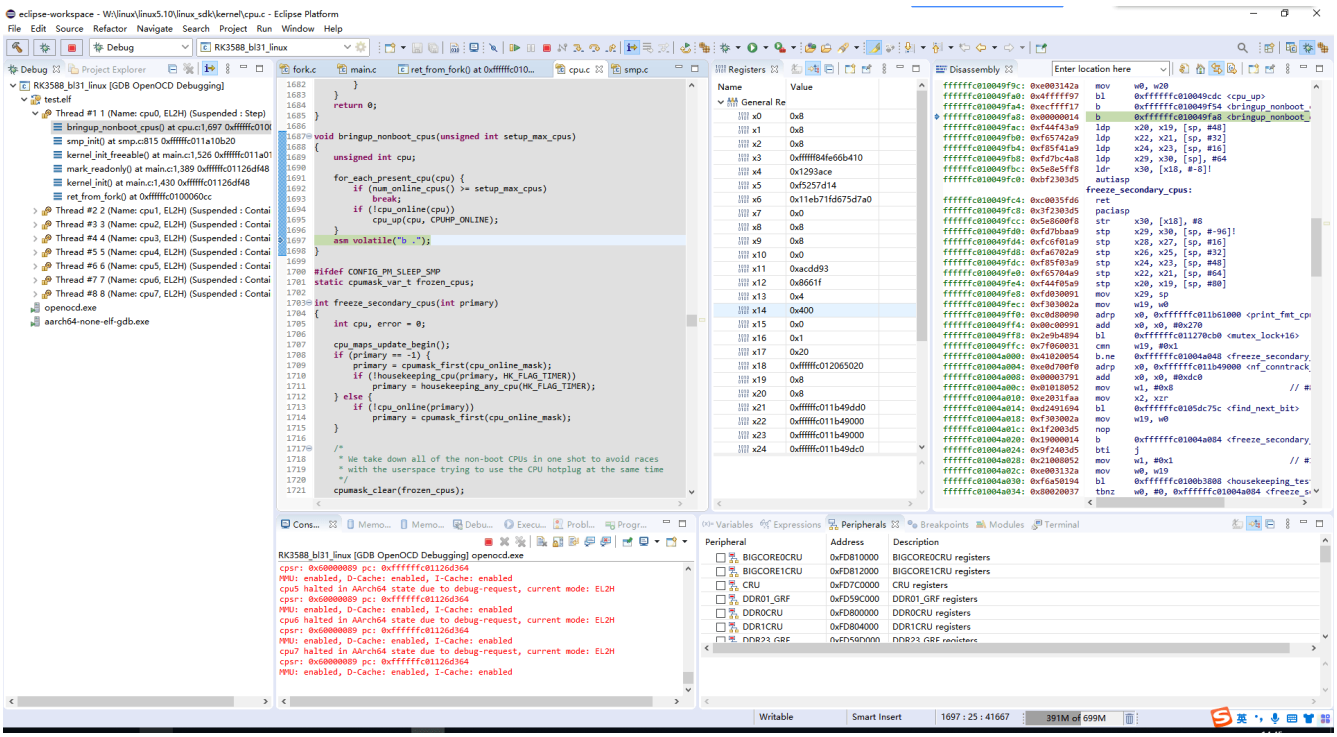
As shown in the figure below, right-click the instruction following the `b` instruction, select **Move To line** to jump to the next instruction, and proceed with single-step debugging.



6.3.2 Kernel Multi-core Phase

```
linux5.10/kernel/cpu.c
void bringup_nonboot_cpus(unsigned int setup_max_cpus)
{
        unsigned int cpu;

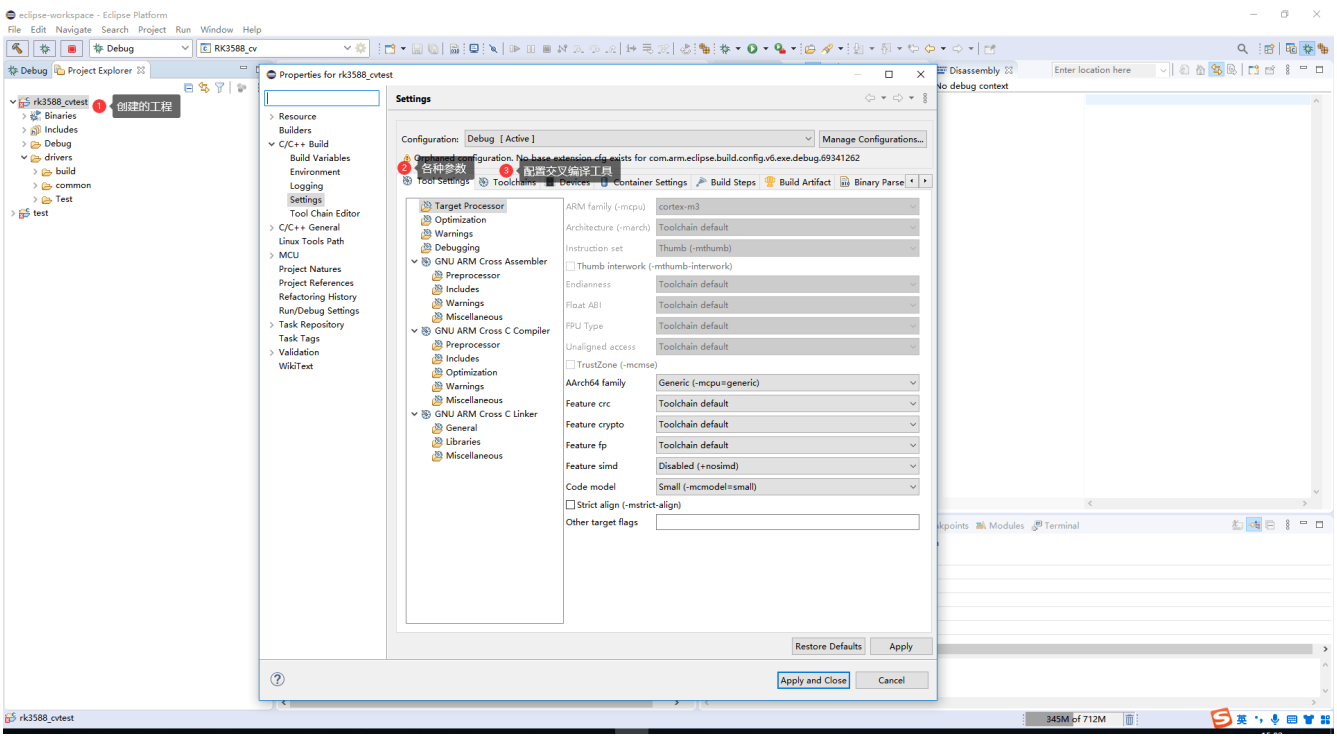
        for_each_present_cpu(cpu) {
                if (num_online_cpus() >= setup_max_cpus)
                        break;
                if (!cpu_online(cpu))
                        cpu_up(cpu, CPUHP_ONLINE);
        }
        asm volatile("b .");
}
```



6.4 Bare-metal Debugging

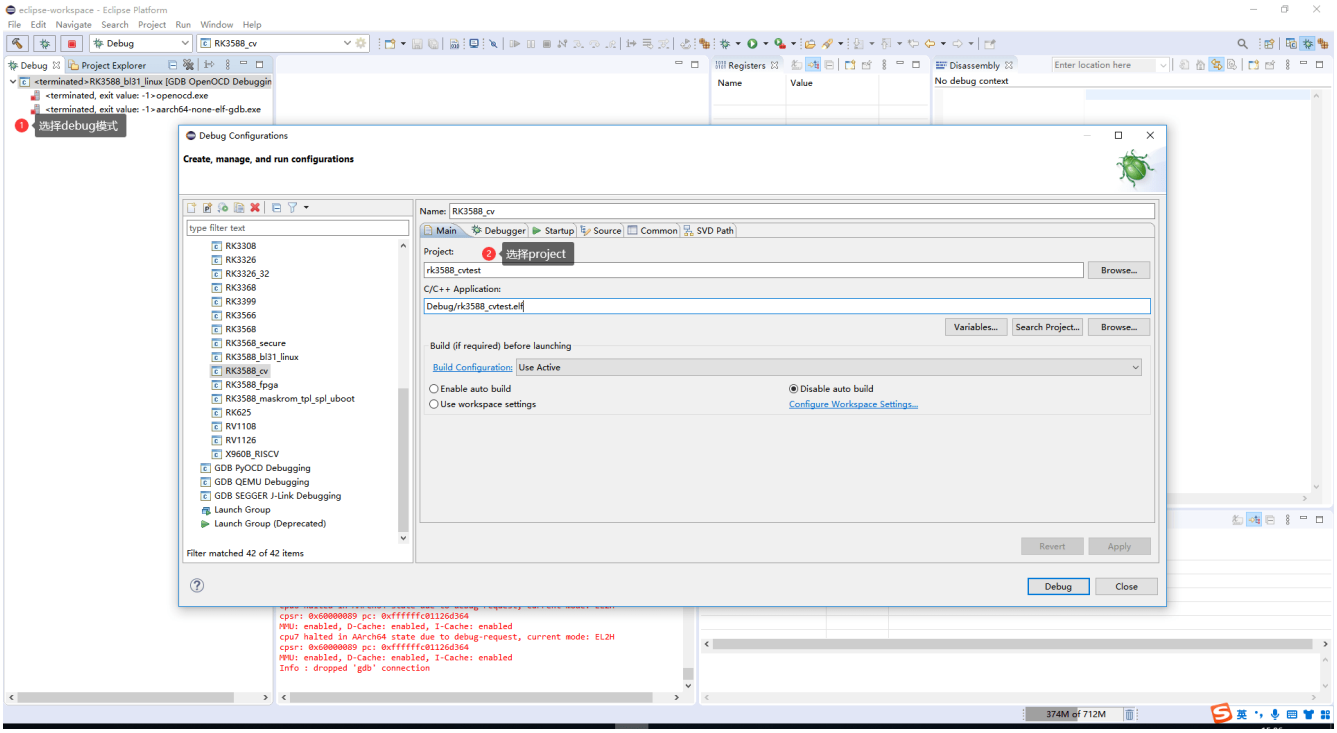
6.4.1 Create Project

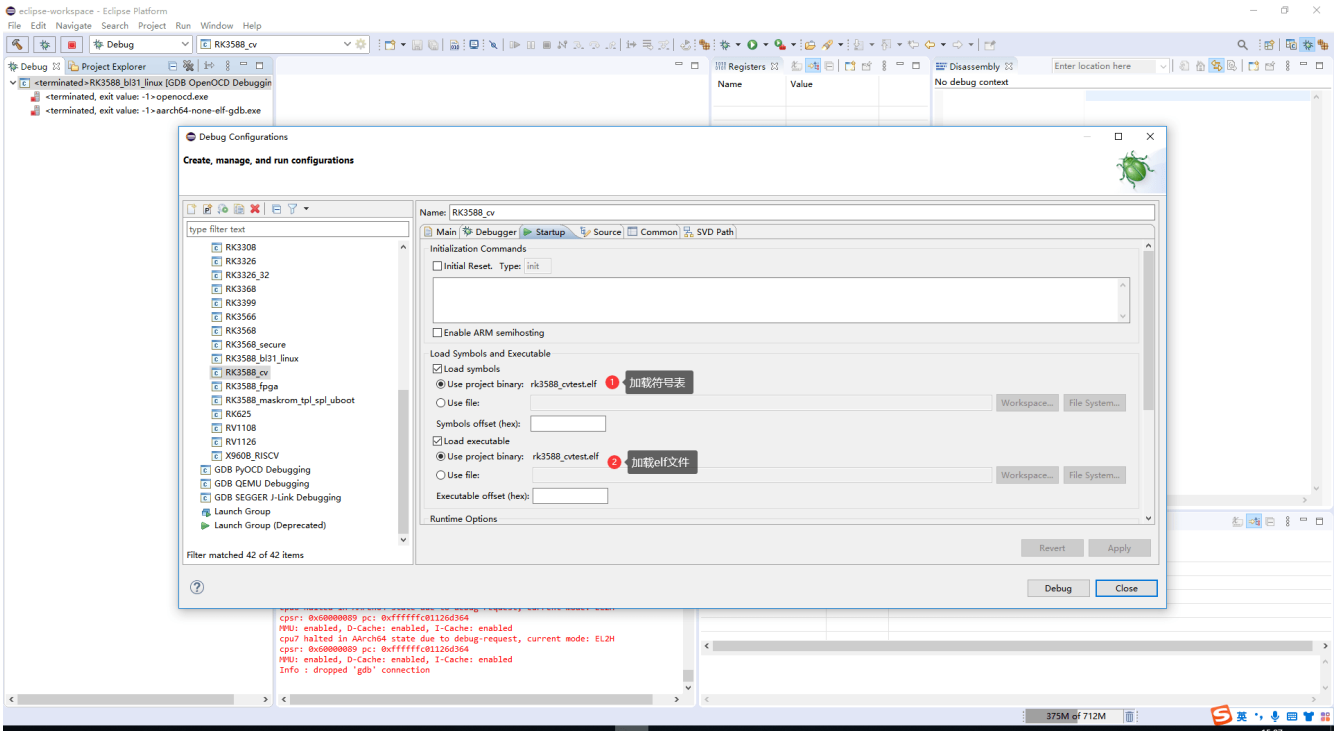
- Create a project using the standard Eclipse installation process
- Configure various compiler parameters
- Configure the cross-compiler



6.4.2 Load and Run Firmware

Note: Copy the configuration file for the target chip you intend to debug, then modify it accordingly.





6.5 Using kgdb

6.5.1 Purpose of kgdb

- Debug a specific Driver
- Trace the entire call stack to identify the source of illegal pointer errors
- Utilize serial communication instead of the JTAG interface for convenience and efficiency

6.5.2 Enabling kgdb

- Configure via menuconfig

```
Device Drivers  --->
   Character devices  --->
      [*]   Virtual terminal
           [*]     Enable character translations in console (NEW)
           [*]     Support for console on virtual terminal (NEW)
-> Kernel hacking
  -> Generic Kernel Debugging Instruments
    [*] KGDB: kernel debugger  --->
      --- KGDB: kernel debugger
      [*]   KGDB: use kprobe blocklist to prohibit unsafe breakpoints (NEW)
      <*>   KGDB: use kgdb over the serial console (NEW)

Kernel hacking  --->
   Debug Oops, Lockups and Hangs  --->
     (0) panic timeout     Set this to 0 to enter kgdb only when a die occurs.
 Alternatively, run `echo 0 > /sys/module/kernel/parameters/panic` to set the panic
timeout to 0, which disables automatic reboot.
```

- Kernel modifications

```
arch/arm64/boot/dts/rockchip/rk3588-android.dtsi
chosen {
        bootargs = "kgdboc_earlycon kgdboc=ttyFIQ0,1500000";
};
```

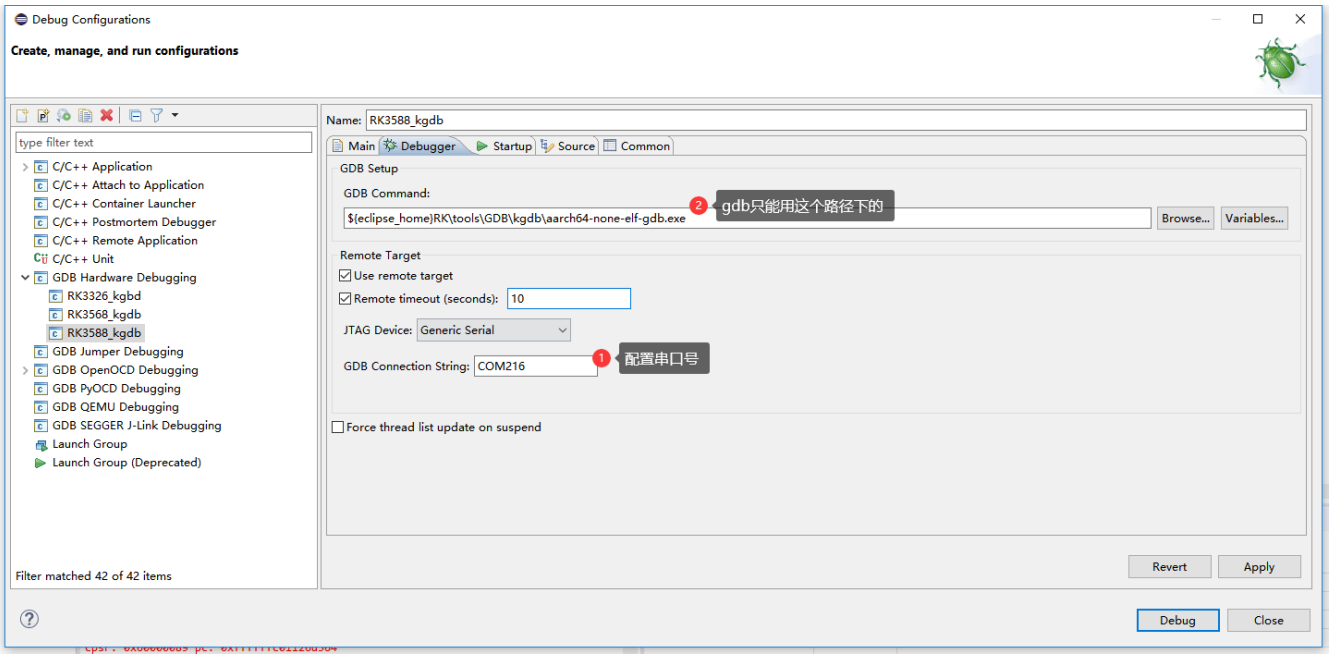
- Verify effectiveness

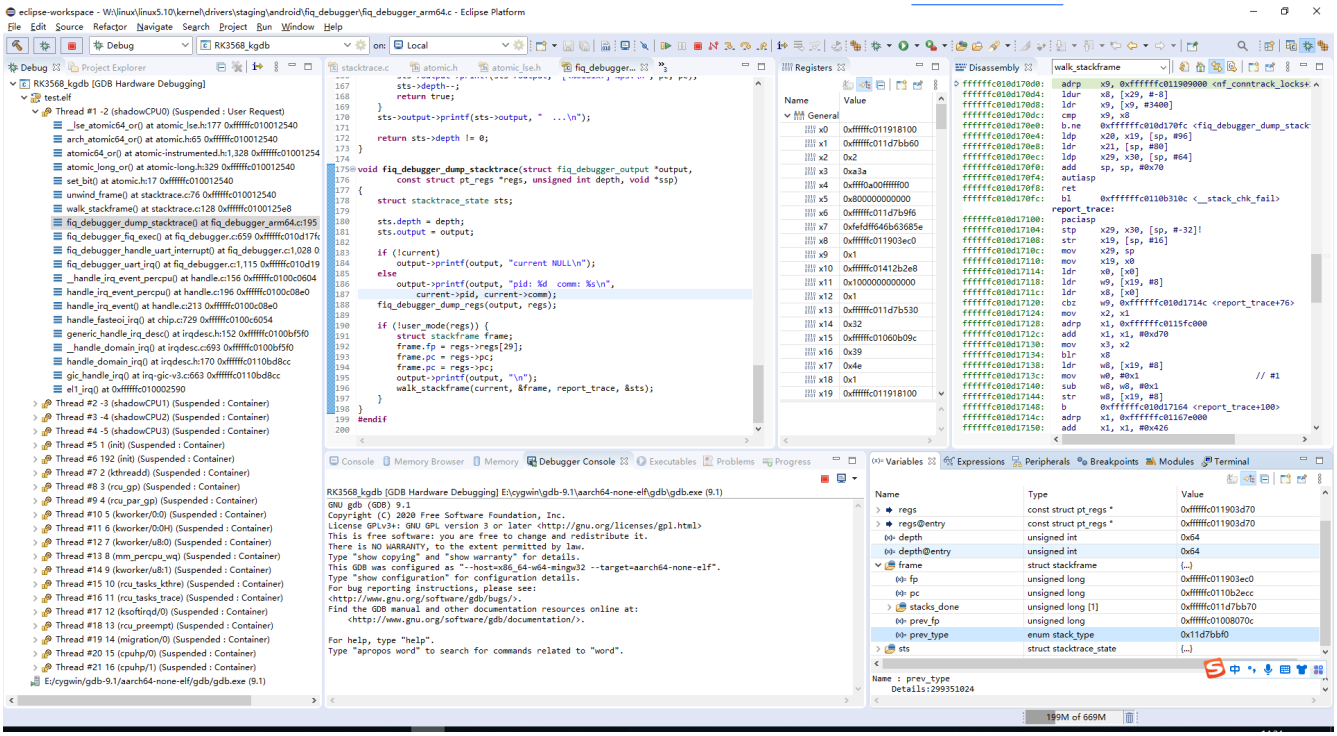
```
echo g > /proc/sysrq-trigger

[    1.385017][    T0] kgdboc: No suitable earlycon yet, will try later
[    1.385069][    T0] earlycon: uart8250 at MMIO32 0x00000000feb50000 (options '')
[    1.390726][    T0] printk: bootconsole [uart8250] enabled
[    2.956193][    T0] kgdboc: Going to register kgdb with earlycon 'uart'
[    2.956196][    T0] KGDB: Registered I/O driver kgdboc_earlycon
[    3.092139][    T1] Unable to handle kernel NULL pointer dereference at virtual
address 000000000000007f
[    3.093010][    T1] Mem abort info:
[    3.093329][    T1]   ESR = 0x96000005
[    3.093672][    T1]   EC = 0x25: DABT (current EL), IL = 32 bits
[    3.094219][    T1]   SET = 0, FnV = 0
[    3.094561][    T1]   EA = 0, S1PTW = 0
[    3.094912][    T1] Data abort info:
```

```
[    3.095238][    T1]   ISV = 0, ISS = 0x00000005
[    3.095651][    T1]   CM = 0, WnR = 0
[    3.095987][    T1] [000000000000007f] user address but active_mm is swapper
[    3.096633][    T1] Internal error: Oops: 96000005 [#1] PREEMPT SMP
[    3.097206][    T1] KGDB: Waiting for remote debugger
```

6.5.3 kgdb Configuration





6.6 Debugging in aarch64 32-bit Mode

-c "cpu0 aarch64_32 "